

1. Why is Python called an interpreted language?

Because Python source code is executed line-by-line by an interpreter (the Python Virtual Machine) at runtime, rather than being compiled entirely into machine code beforehand.

Nuance: Python actually *compiles* to bytecode first (`.pyc`), but that bytecode is then *interpreted* by the PVM—so it's *bytecode-interpreted*, not traditionally compiled to native machine code.

2. What happens internally when you write `print("Hello")`?

1. Lexical analysis – breaks code into tokens.
 2. Parsing – builds an Abstract Syntax Tree (AST).
 3. Compilation – converts AST to bytecode (e.g., `LOAD_NAME`, `CALL_FUNCTION`).
 4. Execution – the Python Virtual Machine (PVM) executes the bytecode, which calls the C function `PyFile_WriteString()` to output "Hello" to `stdout`.
-

3. Why doesn't Python require semicolons?

Python uses newlines as statement terminators by default. The parser treats the end of a logical line as the end of a statement.

Nuance: Semicolons are allowed to separate multiple statements on one line, but it's strongly discouraged by PEP 8.

4. Why is indentation mandatory?

Indentation defines code blocks (instead of curly braces `{}`). The lexer generates `INDENT` and `DEDENT` tokens to mark the start and end of blocks. It's mandatory because it's part of Python's syntax, not just style—it enforces readability.

5. What is the difference between a compiler and an interpreter?

Compiler	Interpreter
Translates entire source code into machine code before execution.	Translates and executes code line-by-line at runtime.
Output is a standalone executable (e.g., <code>.exe</code>).	No separate executable; source is run directly.
Faster execution (machine code).	Slower execution (translation overhead).
Examples: C, C++, Rust.	Examples: JavaScript, Ruby, Python (bytecode-interpreted).

6. Why does Python support multiple programming paradigms?

Python is multi-paradigm because its design offers:

- Procedural – functions and loops.
 - Object-Oriented – classes, inheritance, polymorphism.
 - Functional – `lambda`, `map`, `filter`, first-class functions.
- This flexibility lets developers choose the best approach for the problem.
-

7. What happens when Python executes a file?

1. The interpreter reads the source file.
2. It compiles it to bytecode (`.pyc` is cached if possible).

3. The PVM (Python Virtual Machine) executes the bytecode in a loop.
 4. The `__name__` variable is set to `"__main__"` for the top-level file.
 5. The interpreter runs all statements sequentially (imports, function definitions, etc.).
-

8. What is the purpose of `__name__`?

`__name__` is a built-in special variable that tells you how a module is being used:

- If the script is run directly: `__name__ = "__main__"`.
 - If the script is imported as a module: `__name__ = "module_name"` (the actual filename).
-

9. Why do we use `if __name__ == "__main__":`?

To prevent code from executing when the file is imported into another script. It acts as an entry point guard, allowing the file to be used both as a reusable module and as a standalone program.

10. What is the difference between syntax error and runtime error?

Syntax Error

Runtime Error (Exception)

Occurs during parsing (before execution).

Occurs during execution.

Code cannot run at all.

Code runs until the faulty line is reached.

Example: `print "hi"` (missing parentheses).

Example: `10 / 0` (ZeroDivisionError).

11. Why is Python dynamically typed?

Because type checking is done at runtime, not at compile time. A variable can hold any type, and its type is determined by the value assigned to it. This enables rapid prototyping and flexible code.

12. Can Python variables change data type?

Yes. Since Python is dynamically typed, a variable is just a reference (name) to an object. You can reassign it to a different type at any time:

```
python
x = 5          # int
x = "hello"    # str - perfectly valid
```

13. Why doesn't Python need variable declarations?

Because assignment creates the variable. You don't need to declare a type or allocate memory explicitly. When you write `x = 10`, Python creates a reference `x` to an integer object on the heap automatically.

14. What happens when you assign `a = b`?

- Python evaluates the right-hand side (`b`), which resolves to an object.

- It makes `a` refer to the exact same object that `b` refers to.
 - It does not copy the object. (Assignment is by object-reference, not by value). If `b` is immutable (like `int`), changing `b` later doesn't affect `a`; if mutable (like `list`), both point to the same list.
-

15. Why are keywords reserved?

Keywords (e.g., `if`, `for`, `class`) have fixed, special meanings in Python's syntax. Reserving them prevents ambiguity—you cannot use them as variable/function names, ensuring the parser can correctly identify language constructs.

16. How does Python know where a block starts and ends?

Using indentation and the colon (`:`). The lexer tracks indentation levels and inserts `INDENT` tokens at the start of a block (after a colon) and `DEDENT` tokens when the indentation decreases back to a previous level.

17. Why is Python case-sensitive?

Because identifiers (variable names, function names, class names) are treated as distinct based on their character case. This is a design choice for clarity and consistency (e.g., `myVar` $\neq$ `myvar`). It also aligns with most C-family languages.

18. What happens if indentation is incorrect?

Python raises an `IndentationError` (subclass of `SyntaxError`) at compile time. The program will not run until you fix the inconsistent indentation levels.

19. What is bytecode?

Bytecode is a low-level, platform-independent intermediate representation of your source code. It is a sequence of instructions (like `LOAD_FAST`, `BINARY_OP`) that the PVM understands. It's stored in `.pyc` files for faster startup on subsequent runs.

20. Who executes Python bytecode?

The Python Virtual Machine (PVM) – also called the interpreter loop. It is a loop that reads each bytecode instruction one by one and executes the corresponding C-level logic. The PVM is the core runtime engine of Python.

21. What actually is a variable?

A variable is a named reference (or label) to an object in memory. It is not the data itself—it's a binding stored in a namespace dictionary (e.g., `globals()`, `locals()`). In CPython, it essentially holds the memory address of the object it points to.

22. Where are variables stored?

- Variable names (references) are stored in the stack (inside frame objects—call stack).
 - The actual objects (data) are stored in the heap (dynamic memory).
The stack holds pointers to heap-allocated objects.
-

23. What is an object in Python?

An object is a chunk of heap-allocated memory that contains:

- Value (the actual data).
 - Type (e.g., `int`, `str`, `list`)—determines allowed operations.
 - Reference count (for memory management).
- Everything in Python is an object—even functions, modules, and classes.
-

24. Why can two variables point to the same object?

Because assignment (`a = b`) copies the reference, not the object. This is efficient (no memory duplication) and is fundamental to Python's pass-by-object-reference model. It saves memory and time, especially for large objects.

25. What does `id()` return?

It returns the memory address of the object (in CPython) as an integer. In other implementations (PyPy, Jython), it returns a unique, unchanging integer identifier guaranteed to be distinct for the object's lifetime—but CPython uses the raw address.

26. Why is `is` different from `==`?

`==`

Checks value equality—do the contents match?

Uses the `__eq__()` method.

`is`

Checks object identity—do they point to the *exact same* object in memory?

Compares the result of `id()`.

Example: `[1] == [1]` → `True`.

Example: `[1] is [1]` → `False` (two distinct objects).

Rule of thumb: Use `is` for `None`, booleans, and singletons; use `==` for content comparison.

27. What is object identity?

It's an integer that uniquely identifies an object for its entire lifetime. In CPython, it's the object's memory address. Two objects can never have the same identity at the same time.

28. Why is `None` an object?

Because everything in Python is an object. `None` is a singleton of type `NoneType` that represents "no value". Being an object allows it to be assigned, passed as an argument, returned from functions, and compared using `is`.

29. What happens when an object is no longer used?

Its reference count drops to zero. It becomes garbage and its memory is immediately reclaimed by the reference-counting mechanism (in CPython). If it's part of a cyclic reference, it waits for the cyclic garbage collector.

30. What is Garbage Collection (GC)?

GC is Python's automatic memory reclamation process. In CPython, it's two-tiered:

1. Reference counting (main, immediate).
 2. Generational cyclic collector (backup) – detects and breaks reference cycles (e.g., a list pointing to itself) that ref-counting can't handle.
-

31. Why does Python automatically manage memory?

To free developers from manual allocation/freeing (no `malloc/free`), eliminating common bugs like:

- Memory leaks
- Dangling pointers
- Double frees

This makes Python safer, more productive, and less error-prone.

32. What is reference counting?

A memory management technique where every object maintains a counter of how many references point to it.

- When you create a new reference → counter increments (`Py_INCREF`).
- When you delete a reference → counter decrements (`Py_DECREF`).
- When counter hits 0 → object is deallocated immediately.

Nuance: It's deterministic but has overhead and cannot handle cycles—hence the need for a supplemental GC.

33. Can an object exist without a variable?

Yes. Temporary objects exist without named variables:

- `"hello".upper()` – the string `"hello"` is an object; the result is a new object.
- Function arguments: `func(10)` – the `10` object exists in the stack frame without a user-level variable name.

- Results of expressions: `x = 5 + 3 - 8` exists transiently before assignment.

However, if it has zero references (even temporary), it's deallocated immediately.

34. What happens after `del x`?

- The variable `x` is removed from the local/global namespace.
 - The reference count of the object `x` pointed to is decremented.
 - If the count becomes 0, the object is deallocated immediately (memory freed, `__del__` method called if defined).
 - If the object is part of a reference cycle, it's moved to the GC's pending list for cycle detection.
-

35. Why are immutable objects memory efficient?

- Python interns (caches) small integers and some strings—multiple variables share the same singleton object, saving memory.
 - Since they cannot change, Python can safely use interning and sharing without risk of mutation side-effects.
 - They require less overhead for copy operations (e.g., copying a tuple shares the internal immutable elements).
-

36. Architecture of memory management and PVM?

(Interpreting this as: "Explain the architecture of Python's Memory Management and the PVM.")

Memory Management Architecture (CPython):

- PyObject header – every object starts with a header containing: `ob_refcnt` (reference count) and `ob_type` (pointer to type object).
- Allocator hierarchy:

1. PyMalloc (pymalloc) – optimized allocator for small objects (< 512 bytes) using pools/arenas.
 2. Raw malloc – for large objects.
- Garbage Collector – generational GC (0, 1, 2 generations) to detect and break reference cycles.

PVM (Python Virtual Machine) Architecture:

- Bytecode interpreter – a loop (in `ceval.c`) that reads `PyCodeObject` bytecode instructions (e.g., `LOAD_FAST`, `CALL_FUNCTION`) and executes them via giant switch-case.
- Frame objects – manage local variables, stack, and program counter per function call.
- Runtime state – includes thread state, global interpreter lock (GIL), and module imports.

Creator note: If they ask *who* architected it – Guido van Rossum created Python, but the CPython memory manager and PVM were built by the core development team (with contributions from many).

36. Why is string immutable?

- Hashability – allows strings to be used as dictionary keys and stored in sets (hash must remain constant).
 - Security – prevents unintended side effects (e.g., a string passed to a system call can't be mutated behind your back).
 - Performance – enables interning (caching) and memory sharing; Python can safely reuse string literals.
 - Thread-safety – immutable objects are inherently safe across threads.
-

37. Why is list mutable?

To allow dynamic resizing and in-place modifications (`append`, `pop`, `insert`) without creating a new object every time. This is efficient for collections that grow/shrink frequently during runtime.

38. Why is tuple immutable?

- To be hashable – usable as dict keys and set elements (provided elements are hashable).
 - To guarantee integrity – serves as a fixed record (e.g., coordinates `(x, y)`) that shouldn't change.
 - Performance – immutability allows optimizations in memory allocation and iteration.
-

39. Why use tuple instead of list?

Use Tuple when...

Use List when...

Data is fixed (e.g., days of week, coordinates).

Data changes dynamically (append/remove).

You need a dictionary key or set element.

You don't need hashing.

You want slightly better memory and speed.

You need methods like `sort()`, `reverse()`.

You're returning multiple values from a function (it's idiomatic).

You're storing homogeneous, variable-length data.

40. Why are dictionaries fast?

Because they use a hash table under the hood. Average-case time complexity for lookup, insertion, and deletion is $O(1)$. The hash converts the key directly to a memory index, bypassing linear searches (unlike lists, which are $O(n)$).

41. How does dictionary lookup work?

1. Compute hash – call `hash(key)` to get an integer.
 2. Mask – reduce the hash to fit the table size (e.g., `hash & (size-1)` for power-of-two sizing).
 3. Index – go to that slot in the hash table.
 4. Probe (if collision) – CPython uses open addressing with pseudo-random probing to find the next free/equal slot.
 5. Equality check – once the slot is found, compare `full key == existing_key` to confirm (to handle hash collisions).
-

42. Why can't a list be a dictionary key?

Because lists are mutable (hashable objects must be immutable). If you used a list as a key and later modified it (e.g., `list.append()`), its hash would change, making it impossible for the dictionary to locate the original value. Python solves this by simply making lists unhashable (`TypeError: unhashable type: 'list'`).

43. Why can tuple be a dictionary key?

Tuples are immutable. Once created, their contents cannot change, so their hash value (calculated recursively from their elements) remains constant forever. *Caveat:* A tuple is hashable only if all its elements are hashable (e.g., `(1, [2,3])` is unhashable because the list is mutable).

44. What is hashing?

Hashing is a process that takes arbitrary-sized input (like a string or tuple) and maps it to a fixed-size integer (the hash value) via a deterministic hash function. In Python, it's used to quickly locate objects in hash tables (dicts/sets). The goal is to distribute objects evenly across buckets to minimize collisions.

45. What makes an object hashable?

An object is hashable if:

1. It implements `__hash__()` (which returns an integer).
2. It implements `__eq__()` (for equality comparison).
3. It is immutable (so the hash value never changes during its lifetime).

Examples: `int`, `str`, `tuple` (of hashables), `frozenset`.

Unhashable: `list`, `dict`, `set`, `bytearray`.

46. Why does set remove duplicates?

A set is a hash table that stores only unique keys (with no associated values). When you add an element, Python hashes it; if an equal object already exists in the table (checked via `__eq__`), it's a duplicate and is ignored/overwritten, ensuring mathematical set semantics.

47. How does set detect duplicates?

1. Compute `hash(element)`.
2. Locate the bucket via hashing.
3. If the bucket is empty → new element (not duplicate).
4. If the bucket is occupied → compare `element == existing_element`.

5. If `True` → duplicate (do nothing).
If `False` → collision (handle via probing and check other slots until found or empty).
-

48. Why is frozenset immutable?

To make it hashable, allowing it to be used as a dictionary key or as an element inside another set. It freezes the set's contents at creation—no `add()`, `remove()`, or `update()`.

49. Why use frozenset?

- Use as a key in a dict when you need to group by an unordered collection.
 - Create a set of sets (normal sets can't contain mutable sets).
 - Represent an immutable configuration or an unchanging group of options.
 - Safer for multi-threading (no accidental mutations).
-

50. Difference between set and frozenset?

Feature	<code>set</code>	<code>frozenset</code>
Mutability	Mutable (add/remove/discard)	Immutable
Hashable	 No (cannot be nested in sets/dicts)	 Yes
Creation	<code>{1,2}</code> or <code>set([1,2])</code>	<code>frozenset([1,2])</code>

Methods

Supports `add`, `remove`, `update`

Only supports read-only ops
(`union`, `intersection`, etc.)

51. Why is string ordered?

Strings are sequences of characters stored in a specific index order (0 to n-1). This allows indexing, slicing, and iteration in a predictable order, which is essential for text processing (e.g., `"abc"[1]` always returns `'b'`).

52. Why was dictionary unordered in old Python versions?

Before Python 3.6, dicts used a sparse table where entries were stored in slots determined by hash order. Iteration followed the physical order of these slots, which was essentially random (based on hash values and insertion collisions). There was no separate linked list to track insertion order.

53. Why is dictionary ordered now?

Since Python 3.7 (implementation detail in 3.6), dicts use a compact, memory-efficient layout:

- Two arrays: one indices array (sparse, storing ints) and one entries array (dense, storing key-value pairs in insertion order).
 - Iteration iterates over the dense entries array → preserves insertion order. This was adopted to improve memory usage and performance, and the order-preservation was officially standardized in Python 3.7.
-

54. Difference between shallow copy and deep copy?

Shallow Copy

Creates a new container but populates it with references to the same inner objects.

Changes to inner objects affect both copies.

Done via `list.copy()`, `copy.copy()`, slicing `[:]`.

Example: `a = [[1,2], 3]; b = a.copy();`
`b[0].append(3)` modifies `a`.

Deep Copy

Creates a new container and recursively copies all nested objects (new objects for everything).

Changes to inner objects do NOT affect the deep copy.

Done via `copy.deepcopy()`.

`b[0]` is a completely independent list.

55. Why does `list1 = list2` not create a copy?

Because assignment copies the reference (memory address), not the object. Both variables point to the exact same list object. Any mutation via `list1` reflects in `list2`. To create a copy, you must explicitly use `list2.copy()`, `list2[:]`, or `list(list2)`.

56. Difference between `append()` and `extend()`?

`append(x)`

Adds `x` as a single element at the end (even if `x` is a list).

`extend(iterable)`

Iterates over `iterable` and adds each element individually.

<code>list.append([1,2]) → [[1,2]] (nested).</code>	<code>list.extend([1,2]) → [1,2] (flat).</code>
---	---

Length increases by 1.

Length increases by `len(iterable)`.

57. Why does `sort()` return `None`?

For performance and memory efficiency – it performs an in-place sort (mutates the original list) and returns `None` to signal that mutation occurred. Returning `None` also prevents chaining, reinforcing that the original object was modified. If it returned a new list, it would waste memory.

58. Difference between `sorted()` and `sort()`?

`sort()` (List method)

`sorted()` (Built-in function)

In-place – modifies the original list.

Returns a new list – leaves original unchanged.

Returns `None`.

Returns a sorted list.

Works only on lists.

Works on any iterable (tuples, dicts, generators).

`my_list.sort()`

`new_list = sorted(my_tuple)`

59. Why does tuple consume less memory?

- Tuples are fixed-size – Python allocates exactly the required memory upfront, with no extra capacity (no overallocation).
 - Lists allocate extra capacity (overallocation) to accommodate future appends efficiently (e.g., ~12.5% spare slots).
 - Tuple structs in CPython are also slightly smaller (fewer fields to maintain) compared to list objects.
-

60. Which datatype is fastest for searching?

`set` (and `dict` keys) – average $O(1)$ lookup due to hashing.

- `list/tuple`: $O(n)$ (linear scan).
- `set/frozenset`: $O(1)$ average.

Nuance: If you need to search many times, convert to a set first. If your data is sorted, you can use `bisect` on a list for $O(\log n)$, but raw membership is fastest on `set`.

61. Why is string immutable? (*Duplicate from Q36—Here's the concise recap*)

- Hashability – allows use as dict keys and in sets.
- Security – prevents side effects (e.g., a string passed to an OS call can't be mutated).
- Performance – enables interning (caching) and memory sharing across variables.
- Thread-safety – no locking needed for read-only access across threads.

62. Why does slicing create a new object?

Because strings are immutable. You cannot modify the original string in-place. When you slice (`s[1:3]`), Python allocates a new string object in memory and copies the selected characters into it. This guarantees that the original remains unchanged and the new slice is a fully independent object.

63. Difference between `find()` and `index()`?

`find(sub)`

Returns the lowest index where `sub` is found.

Returns `-1` if not found.

Safer for scanning (no exception handling overhead).

`index(sub)`

Returns the lowest index where `sub` is found.

Raises `ValueError` if not found.

Preferred when you expect the substring to exist (explicit is better than implicit).

64. Difference between `replace()` and `translate()`?

`replace(old, new, count)`

Replaces whole substrings with another substring.

`translate(table)`

Replaces individual characters (1-to-1 or 1-to-many) using a translation table.

Example: `"hello".replace("l", "x")` → `"hexxo"`.

Example: `"hello".translate(table)`
(maps `'h' → 'H'`, `'e' → 'E'`).

Works on overlapping patterns? No
(sequential).

Works on any character mapping defined in
the table.

`replace()` is simpler for word-level
changes.

`translate()` is much faster for
character-level bulk replacements because
it runs at C speed in a single pass.

65. Why use `join()` instead of `'+'` for concatenation?

- Strings are immutable – each `+` creates a new string object, copying all characters from both operands.
 - In a loop (`s += x`), this results in $O(n^2)$ time complexity (Schlemiel the Painter's algorithm).
 - `''.join(list)` pre-computes the total size, allocates memory once, and copies each string exactly once → $O(n)$.
 - `join()` is also more memory-efficient and significantly faster for large collections.
-

66. Difference between `split()` and `partition()`?

`split(sep, maxsplit)`

`partition(sep)`

Splits the string at all occurrences (or up to
`maxsplit`).

Splits the string at the first occurrence only.

Returns a list of all pieces.

Returns a tuple of 3 elements: `(head, separator, tail)`.

If sep not found → returns `[whole_string]`.

If sep not found → returns `(whole_string, "", "")`.

Example: `"a,b,c".split(",")` → `['a', 'b', 'c']`.

Example: `"a,b,c".partition(",")` → `('a', ',', 'b,c')`.

Use `split()` for all tokens; use `partition()` for a single split (e.g., parsing key-value pairs).

67. Why use `format()` instead of concatenation?

1. Readability – separates the template from the data: `"Hello {}".format(name)`.
 2. Type-safety – automatically handles `__str__` vs `__repr__`, and formatting specifiers (e.g., `{:.2f}`).
 3. Performance – `format()` is typically faster than many `+` operations because it creates the final string in one step.
 4. Prevents errors – no manual casting of non-strings to `str` (it's done internally).
-

68. What are f-strings? (Python 3.6+)

Formatted string literals – strings prefixed with `f` (e.g., `f"Hello {name}"`). They allow inline evaluation of expressions inside `{}` at runtime. They are the modern, recommended way for string formatting in Python.

69. Why are f-strings faster?

- They are evaluated at runtime but the expressions inside `{}` are evaluated in the local/global scope directly, with minimal overhead.
 - Unlike `format()` or %-formatting, f-strings do not involve method calls or dictionary lookups for the placeholders; the parsing and code generation happen at compile time (in the bytecode, they become specialized `FORMAT_VALUE` and `BUILD_STRING` ops).
 - They are generally 2–3x faster than `format()` and %-formatting.
-

70. Difference between `strip()`, `lstrip()`, `rstrip()`?

Method	Removes from...
<code>strip(chars)</code>	Both left (leading) and right (trailing) ends.
<code>lstrip(chars)</code>	Only the left/leading end.
<code>rstrip(chars)</code>	Only the right/trailing end.
<i>Default:</i> all remove whitespace (spaces, <code>\n</code> , <code>\t</code> , etc.) if no <code>chars</code> argument is given.	

71. Why does string support slicing?

Because strings are sequences (ordered collections of characters). The sequence protocol requires `__getitem__` which supports slicing via `slice` objects. Slicing allows efficient access to substrings without writing manual loops, making text processing intuitive and concise.

72. What happens internally during slicing?

1. A `slice` object is created (e.g., `slice(1, 5, 2)`).
 2. Python's C-level `sq_slice` (or `mp_subscript`) calculates the actual indices based on the sequence length.
 3. It determines the number of elements (via `PySlice_GetIndicesEx`).
 4. It allocates a new string object of exactly that size.
 5. It copies the characters from the original string into the new memory using `memcpy` (in C) for speed.
 6. The new string object is returned.
-

73. Can string slicing modify the original string?

Absolutely not. Strings are immutable. Slicing always returns a new string. The original string remains untouched in memory (unless its reference count drops to zero and it's garbage collected).

74. Why is `[::-1]` used for reverse? (Correction: it's `[::-1]`)

- `[::-1]` is the idiomatic way to reverse a string in Python.
 - Start: empty (default = beginning).
 - Stop: empty (default = end).
 - Step: `-1` – tells the slice to move backwards by 1 character each step.
 - Internally, Python creates a new string by iterating from the last index to the first, effectively reversing the sequence.
 - Why not `[::1]`? That's just the identity slice (returns a shallow copy of the string).
-

75. Difference between `capitalize()`, `title()`, `upper()`?

Method	Operation	Example (<code>"hello world"</code>)
<code>capitalize()</code>	Converts first character to uppercase, all other characters to lowercase.	<code>"Hello world"</code>
<code>title()</code>	Converts first character of every word to uppercase, others to lowercase.	<code>"Hello World"</code>
<code>upper()</code>	Converts all characters to uppercase.	<code>"HELLO WORLD"</code>

Nuance: `title()` uses a language-agnostic algorithm that considers apostrophes (e.g., `"don't".title()` → `"Don' T"` – watch out for that trap!).

76. Difference between `==` and `is`? (Quick recap)

`==`

Checks value equality – do the contents match?

Uses `__eq__()` method.

Example: `[1] == [1]` → `True`.

`is`

Checks object identity – do they point to the exact same memory address?

Compares `id()` values.

Example: `[1] is [1]` → `False`.

77. Why does `5 == 5.0` return `True`?

Because `==` checks value equality, not type. Python's numeric types follow numeric coercion rules – the integer `5` and float `5.0` represent the same mathematical value, so they compare equal. Under the hood, Python's `__eq__` for numeric types converts integers to floats (or uses rational comparison) before comparing.

78. Why is `True == 1`?

Because `bool` is a subclass of `int` in Python. `True` is literally the integer `1` with a different type label, and `False` is `0`. This heritage exists for historical reasons (pre-boolean Python used `0/1`) and allows things like `sum([True, False, True])` → `2`.

79. Difference between `and` and `&`?

`and` (Logical AND)

Boolean/logical operator – returns the first falsy operand or the last truthy operand (short-circuits).

Short-circuits: `False and x` → doesn't evaluate `x`.

Works on any type (truthiness evaluated).

Example: `5 and 3` → `3` (truthy last).

`&` (Bitwise AND)

Bitwise operator – performs a bitwise AND on integer operands, or set intersection on sets.

Does not short-circuit – evaluates both operands.

Primarily works on numeric types (int, bool) and sets.

Example: `5 & 3` → `1` (`101 & 011 = 001`).

 **Gotcha:** For `bool` operands, `True & False` → `False`, but never use `&` for logical conditions – it doesn't short-circuit and can cause unexpected evaluation order.

80. Difference between `or` and `|`?

`| or` (Logical OR) | `|` (Bitwise OR) |

`| :--` | `| :--` |

| Boolean/logical – returns the first truthy operand or the last falsy operand (short-circuits). | Bitwise – performs bitwise OR on integers, or set union on sets. |

| Short-circuits: `True or x` → doesn't evaluate `x`. | Does not short-circuit. |

| Works on any type. | Primarily numeric types and sets. |

| Example: `0 or 5` → `5` (first truthy). | Example: `1 | 2` → `3` (`01 | 10 = 11`). |

81. What is short-circuit evaluation?

A strategy where Python stops evaluating a boolean expression as soon as the final result is determined, without evaluating the remaining operands:

- **and**: If the left operand is `False`, the result is `False` regardless of the right → right is not evaluated.
- **or**: If the left operand is `True`, the result is `True` regardless of the right → right is not evaluated.

Example: `x = 5 / 0` on the right of `False and ...` never runs, preventing a `ZeroDivisionError`.

82. Why does Python evaluate left to right?

1. Predictability – deterministic evaluation order makes code easier to reason about.
 2. Short-circuiting – only works if evaluation is left-to-right (the left operand is assessed first).
 3. Consistency – follows the standard convention of most programming languages (C, Java, JavaScript), making it familiar.
 4. Operator precedence – still applies (e.g., `and` has lower precedence than `or`), but operands for each operator are evaluated left-to-right.
-

83. Difference between membership and identity operators?

Membership (`in`, `not in`)

Identity (`is`, `is not`)

Checks if a value exists in a collection (or if a substring exists in a string).

Checks if two variables refer to the exact same object (same memory address).

Uses `__contains__()` (or falls back to iteration).

Compares `id()` (memory address).

Example: `3 in [1,2,3]` → `True`.

Example: `a = [1,2]; b = a; a is b` → `True`.

Focus on containment.

Focus on identity/reference.

84. Why use `in`?

- Readability – it's Pythonic and reads like plain English: `if value in container:`.
 - Abstraction – you don't need to manually loop or check indices; it leverages the object's `__contains__` for optimized containment checks.
 - Performance – for `set` and `dict`, it's $O(1)$ (hash-based); for `list/tuple`, it's $O(n)$. It automatically uses the best implementation available for the container type.
 - Versatility – works on strings (substring check), lists, tuples, sets, dicts (checks keys), and custom objects implementing `__contains__`.
-

85. What happens internally during comparison (`a == b`)?

1. Python calls `a.__eq__(b)` (the left operand's `__eq__` method).
2. If `__eq__` returns `NotImplemented` (instead of `True/False`), Python then calls `b.__eq__(a)` (the right operand's `__eq__` method).
3. If `b.__eq__(a)` also returns `NotImplemented`, Python falls back to identity comparison (i.e., `a is b`) as a last resort.
4. For ordered comparisons (`<`, `>`, `<=`, `>=`), a similar process happens using `__lt__`, `__gt__`, etc., but if those return `NotImplemented`, Python raises `TypeError` (unless rich comparison fallbacks exist).

For `is`: It's a direct memory address comparison in C (`PyObject_RichCompareBool` with `Py_EQ` for identity). No method calls are involved.

86. Why do we create functions?

- Reusability – write once, use many times (DRY principle).
 - Modularity – break complex problems into smaller, manageable pieces.
 - Abstraction – hide implementation details; callers only need to know the interface.
 - Testability – isolated units are easier to test and debug.
 - Readability – well-named functions act as documentation for what the code does.
-

87. What is the advantage of functions?

1. Code reuse – avoid duplicating logic.
 2. Easier maintenance – changes only need to be made in one place.
 3. Reduced complexity – decompose large programs into smaller, understandable chunks.
 4. Namespace isolation – variables inside functions are local, preventing naming conflicts.
 5. Easier debugging and testing – functions can be unit-tested independently.
-

88. Difference between parameter and argument?

Parameter

Argument

Variable listed in the function definition (the placeholder).

Actual value passed to the function when calling it.

Example: `def func(x, y):` → `x` and `y` are parameters.

Example: `func(5, 10)` → `5` and `10` are arguments.

89. Why use default arguments?

- To provide sane default values, making parameters optional.
 - Reduces the amount of boilerplate code (callers can omit common values).
 - Allows backward compatibility when adding new parameters to existing functions.
 - Example: `print("hi", end="\n")` – `end` defaults to newline, but you can override it.
-

90. Why are mutable default arguments dangerous?

Default argument values are evaluated only once at function definition time (not each call). If the default is mutable (e.g., `list`, `dict`), and you mutate it inside the function, that mutation persists across all subsequent calls.

Classic trap:

```
python
```

```
def append_to_list(value, lst=[]):  
    lst.append(value)  
    return lst
```

```
print(append_to_list(1)) # [1]
print(append_to_list(2)) # [1, 2] <- Surprise!
```

Fix: Use `None` as default and create a new mutable object inside:

python

```
def append_to_list(value, lst=None):
    if lst is None:
        lst = []
    lst.append(value)
    return lst
```

91. What is `*args`?

- `*args` (arguments) collects any number of positional arguments passed to a function into a tuple.
 - The name `args` is convention; the `*` is what matters.
-

92. What is `**kwargs`?

- `**kwargs` (keyword arguments) collects any number of keyword arguments passed to a function into a dictionary.
 - The name `kwargs` is convention; the `**` is what matters.
-

93. Why use `*args`?

- To create functions that accept a variable number of positional arguments (e.g., `sum(*args)`, `max(*args)`).
 - To forward/pass-through positional arguments to another function without explicitly naming each one (common in decorators and wrappers).
-

94. Why use `**kwargs`?

- To accept a variable number of keyword arguments (e.g., for configuration dictionaries).
 - To forward/pass-through keyword arguments to another function, especially useful in decorators and subclassing (e.g., `super().__init__(**kwargs)`).
-

95. Difference between local and global variables?

Local Variable

Global Variable

Defined inside a function.

Defined at the module level (outside any function).

Accessible only within that function.

Accessible from anywhere in the module.

Exists only while the function runs.

Exists for the entire program lifetime.

Can be read in a function, but to modify it you need the `global` keyword.

Modifiable anywhere (but use `global` to modify inside functions).

96. What is the LEGB rule?

The order Python uses to resolve variable names:

- Local – variables inside the current function.
- Enclosing – variables in any enclosing functions (closures, nested scopes).
- Global – variables at the module level.
- Built-in – Python's built-in names (e.g., `print`, `len`, `open`).

Python searches LEGB in that sequence and stops at the first match.

97. Why does Python search variables in LEGB order?

1. Efficiency – local variables are accessed fastest (closest to the CPU, stored in a small array).
 2. Security/isolation – inner scopes can use names without worrying about outer-scope conflicts; local modifications don't pollute the global namespace.
 3. Clarity – follows the principle of least surprise; most common usage (local) is checked first.
-

98. What is recursion?

A technique where a function calls itself to solve a smaller instance of the same problem. It requires:

1. Base case – a stopping condition.

2. Recursive case – the function calls itself with modified arguments, moving toward the base case.

Example: `factorial(n) = n * factorial(n-1)` with `factorial(0) = 1`.

99. Why recursion uses more memory?

Each recursive call adds a new stack frame to the call stack, containing:

- Local variables
- Return address
- Function parameters

These frames accumulate until the base case is reached. Deep recursion $\rightarrow O(n)$ memory (where n is recursion depth). In Python, this can hit the recursion limit (`sys.getrecursionlimit()`, default ~ 1000) and raise `RecursionError`. Iterative solutions use $O(1)$ memory.

100. What is a lambda?

An anonymous function defined with the `lambda` keyword: `lambda args: expression`

- Has no name (unless assigned to a variable).
- Can only contain a single expression (no statements).
- Implicitly returns the result of the expression.

Example: `square = lambda x: x**2` (though a normal function is often clearer).

101. Why lambda has only one expression?

- Design philosophy – lambdas are intended for simple, inline operations (e.g., as arguments to `map`, `filter`, `sorted`).
- They are not meant to replace full functions with complex logic.

- Allowing statements would make them as complex as regular functions, defeating their purpose of brevity.
 - For complex logic, Python encourages writing a named function for readability and reusability.
-

102. Difference between lambda and normal function?

Lambda

Normal Function (def)

Anonymous (no intrinsic name).

Has a name.

Single expression only (no statements).

Can have multiple statements.

Implicit `return` (returns expression).

Needs explicit `return` to return a value.

No docstring support.

Supports docstrings (`"""..."""`).

Cannot have type annotations (Python 3.9+ allows minimal ones, but limited).

Full annotation support.

Example: `lambda x: x+1`.

Example: `def add(x): return x+1`.

103. What are higher-order functions?

Functions that do at least one of the following:

1. Take one or more functions as arguments (e.g., `map`, `filter`, `sorted(key=...)`).
2. Return a function as its result (e.g., decorators, closures).

They enable functional programming patterns in Python.

104. Why are functions first-class objects?

Because functions in Python:

- Can be assigned to variables.
- Can be stored in data structures (lists, dicts).
- Can be passed as arguments to other functions.
- Can be returned from other functions.
- Have attributes (e.g., `__name__`, `__doc__`).

Under the hood: Everything in Python is an object, including functions (which are instances of the `function` class).

105. Can a function return another function?

Absolutely yes. This is how closures and decorators work.

Example:

```
python
```

```
def outer(msg):  
    def inner():
```

```

        return f"Hello, {msg}"

    return inner # returns the function object

func = outer("World")

print(func()) # "Hello, World"

```

The returned function `inner` captures the enclosing scope's `msg` variable (a closure).

🎯 Final Interview Pro-Tips for this section:

- Q90 (mutable defaults) – this is a *favorite* interview trap. Always demonstrate the fix with `None`.
- Q91/92 (`*args/kwargs`)^{**} – mention they are conventionally named `args/kwargs` but the `*/**` is what matters. Also mention *argument unpacking* (`*list`, `**dict`) when calling functions.
- Q96 (LEGB) – give a concrete example like:
- python

```

x = "global"

def outer():
    x = "enclosing"

    def inner():
        x = "local"

```

- `print(x)` # prints "local"
- Q104/105 – mention *closures* and how they capture variables, which is the foundation for decorators and functional patterns.

106. Why do we need OOP?

- Modeling real-world entities – maps business logic to tangible objects (e.g., `Customer`, `Order`).
 - Code reusability – inheritance and composition reduce duplication.
 - Modularity & maintainability – encapsulate state and behavior; changes are isolated.
 - Scalability – large codebases are easier to manage with clear boundaries.
 - Security – encapsulation restricts direct access to internal data.
 - Polymorphism – write generic code that works with diverse object types through a common interface.
-

107. Difference between class and object?

Class	Object
A blueprint or template for creating objects.	An instance of a class (concrete entity).
Defined once using <code>class</code> .	Created multiple times using <code>ClassName()</code> .
Does not occupy memory (at runtime, it's a type object).	Occupies heap memory.
Contains method definitions and class variables.	Contains instance-specific state (<code>self</code> attributes).

108. Why create objects?

- To hold state (instance variables) unique to each entity (e.g., different `Employee` objects have different `name`, `salary`).
- To invoke behavior (methods) that operates on that specific state.

- Each object has its own namespace, allowing multiple instances to coexist without interference.
-

109. What happens when an object is created (`MyClass()`)?

1. `__new__` is called (static method) – allocates memory and returns a new instance (the actual constructor).
 2. `__init__` is called (initializer) – receives the instance as `self` and sets up attributes.
 3. The instance is returned to the caller.
 4. The instance's reference count starts at 1.
-

110. What is a constructor?

- In Python, `__new__` is the *actual* constructor (allocates memory), but colloquially, `__init__` is called the constructor because it's the method users define to initialize the object.
 - `__init__` is an initializer, not a constructor.
-

111. Why do we need a constructor (`__init__`)?

- To initialize the object's state (set initial attribute values).
 - To ensure the object is in a valid, usable state immediately after creation.
 - To perform any setup (e.g., opening files, establishing DB connections, dependency injection).
-

112. What happens if a constructor is not defined?

- Python uses the default `__init__` inherited from `object`, which does nothing.

- The object is still created, but it has no instance attributes (unless you add them later).
 - No error occurs – the object just starts empty.
-

113. Why is the constructor called automatically?

Because Python's object creation protocol (`__call__` on the class metaclass) explicitly invokes `__new__` followed by `__init__` as part of the instantiation workflow. It's built into the language's machinery to guarantee every object gets a chance to initialize.

114. Why is `self` required?

- To reference the specific instance being operated on.
 - Instance methods need to know *which* object's attributes to read/write.
 - Without `self`, Python wouldn't know which instance's data to use.
-

115. Is `self` a keyword?

No. It's a convention (enforced by PEP 8). You can use `this`, `me`, or any other name—but always use `self` for readability and to avoid confusing other developers.

116. What does `self` actually store?

- A reference (memory address) to the current object instance.
 - It is passed implicitly by Python when you call a method on an instance (e.g., `obj.method()` → `method(obj, ...)`).
-

117. Why use `__init__()` instead of another method?

- `__init__` is a special (dunder) method that Python automatically calls during instantiation.
 - If you used another name (e.g., `initialize`), you'd have to call it explicitly after creation, which is error-prone and non-idiomatic.
-

118. What do double underscores (`__`) mean?

- Leading double underscores (no trailing) → name mangling (makes the attribute "private" to avoid accidental overriding in subclasses).
 - Leading and trailing double underscores (`__init__`, `__str__`) → dunder/magic methods reserved for Python's internal protocol (e.g., operator overloading, object lifecycle).
-

119. What is name mangling?

- Python renames attributes that start with `__` (but not `__`) by adding the class name: `_ClassName__var`.
 - This prevents accidental collisions when a subclass defines an attribute with the same name.
 - *Example:* In class `A`, `self.__x` becomes `self._A__x` internally.
-

120. Difference between `_var`, `__var`, and `__var__`?

Convention

Meaning

<code>_var</code>	"Protected" (internal use). A convention indicating "private, but don't enforce it".
<code>__var</code>	"Private" (name mangled to <code>_ClassName__var</code>). Strongly discourages external access.
<code>__var__</code>	Magic method (dunder). Reserved for Python's internal protocol. Avoid creating your own dunder.

121. Why use private variables?

- To encapsulate internal state and protect invariants.
 - To provide a controlled interface via getters/setters or `@property` – allowing validation or side-effects on access.
 - To avoid breaking client code if the internal representation changes.
-

122. Can private variables really be private?

No. Python follows the "We are all consenting adults" philosophy. Name mangling provides name obfuscation, not true access control. You can still access `obj._ClassName__var` directly. It's a convention to discourage, not prevent, access.

123. What is encapsulation?

The bundling of data (attributes) and methods (behaviors) that operate on that data into a single unit (class), and restricting direct access to the internal state (using private variables and public methods/properties). It enforces data hiding and promotes modularity.

124. Why is inheritance useful?

- Code reuse – child classes automatically get parent methods/attributes.
 - Hierarchical classification – models "is-a" relationships (e.g., `Dog` inherits from `Animal`).
 - Polymorphism – allows substituting subclasses in code that expects the parent type.
 - Extensibility – override or add functionality without modifying existing code.
-

125. Why use polymorphism?

- Write generic code that works with objects of different classes through a common interface (e.g., `animal.speak()` works for `Dog`, `Cat`, `Cow`).
 - Dependency injection and strategy patterns become possible.
 - Promotes loose coupling and open/closed principle (open for extension, closed for modification).
-

126. Difference between method overriding and overloading?

Overriding	Overloading
Runtime polymorphism – a subclass provides a specific implementation of a method already defined in its superclass.	Compile-time (static) polymorphism – multiple methods with the same name but different parameters (number/type).
One method per class with same signature.	Multiple methods with same name but different signatures.

Example: `class Dog: def speak(self):`
`... overrides Animal.speak.`

Example: `add(a, b)` and `add(a, b, c)`
(not supported natively).

127. Does Python support method overloading?

Not natively. Python does not support multiple methods with the same name and different parameters. However, you can simulate it using:

- Default arguments (`def func(a, b=None)`).
 - `*args` and `**kwargs` to handle variable arguments.
 - `functools.singledispatch` for single-dispatch generic functions.
-

128. What is method resolution order (MRO)?

- The order in which Python searches for methods/attributes in a class hierarchy (especially with multiple inheritance).
 - Python uses the C3 linearization algorithm to compute a monotonic, deterministic order.
 - You can view it via `ClassName.__mro__` or `ClassName.mro()`.
 - *Example:* `D` inherits from `B` and `C`, which inherit from `A` → MRO ensures parent classes are searched left-to-right, depth-first, but avoids repeating ancestors.
-

129. What is `super()`?

- A built-in function that returns a proxy object that delegates method calls to the next class in the MRO (usually the parent or sibling class).
 - It is not the "parent" – it follows the MRO, which is crucial for cooperative multiple inheritance.
-

130. Why use `super()`?

- To call parent class methods (e.g., `super().__init__()`) cleanly, without hardcoding the parent class name.
 - Crucial for multiple inheritance – it ensures each parent is called exactly once (avoids diamond problem issues).
 - Makes code maintainable – if the parent class changes, you don't need to update hardcoded names.
-

131. What happens if `super()` isn't used?

- The parent's `__init__` is not automatically called – you must call it explicitly using the parent class name: `Parent.__init__(self)`.
 - In multiple inheritance, this leads to repeated calls to the same parent (diamond problem) and potentially incorrect initialization order, causing bugs and hard-to-debug issues.
-

132. What is abstraction?

The concept of hiding complex implementation details and exposing only the essential features (interface) to the user. It reduces complexity by providing a clear, high-level API and ignoring irrelevant details. In Python, implemented via abstract classes (ABC module) and abstract methods.

133. Why use abstract classes?

- To define a common interface that all subclasses must implement.
- To prevent instantiation of the base class (it's only meaningful as a template).
- To enforce that specific methods are implemented in subclasses (using `@abstractmethod`).
- To provide partial implementation – some concrete methods can be shared.

134. Difference between interface and abstract class?

(Note: Python doesn't have a native interface keyword; we use `ABC` for both.)

Abstract Class	Interface (conceptual)
Can have both abstract and concrete methods.	Typically only method signatures (no implementation).
Can have state (instance attributes/class variables).	Usually stateless (no attributes).
Can have a <code>__init__</code> .	No constructor (or only for DI).
Supports single inheritance (though can mix with interfaces in some languages).	Supports multiple inheritance (in languages that have interfaces).
Example: <code>Animal</code> with <code>speak()</code> abstract and <code>eat()</code> concrete.	Example: <code>Flyable</code> with <code>fly()</code> abstract only.

135. When should composition be preferred over inheritance?

- "Has-a" relationship vs. "Is-a" (e.g., `Car` has an `Engine` rather than `Car` is an `Engine`).
- When the hierarchy is deep and fragile (inheritance leads to tight coupling and the "fragile base class" problem).
- When you need to change behavior dynamically at runtime (composition allows swapping components).
- To avoid the diamond problem in multiple inheritance.

- To keep classes focused – using composition delegates specialized behavior to dedicated helper classes.

Rule of thumb: "Favor composition over inheritance" (GoF design principle) – use inheritance only when there's a genuine *is-a* relationship and the base class is truly a generalization.

🎯 Interview Pro-Tips for this section:

- Q114 (self) – many interviewers expect you to say *"self is required because Python passes the instance explicitly"*.
- Q118/120 – show the table for `_`, `__`, and `___`; they love this distinction.
- Q128 (MRO) – mention *C3 linearization* and give a diamond inheritance example – it demonstrates deep knowledge.
- Q129/130 (super) – emphasize *cooperative multiple inheritance* and that `super()` follows MRO, not just the parent.
- Q135 (composition vs inheritance) – give a real-world example like *delegating logging to a logger object* rather than inheriting from a `Logger` class.

8. Exception Handling

136. Why do exceptions occur?

- Invalid input (e.g., dividing by zero, non-integer conversion).
- External resource failures (file not found, network timeouts, database unavailability).
- Programming errors (index out of range, key not found, attribute not present).
- Exceptions are Python's way of signaling that something unexpected happened during execution.

137. Difference between error and exception?

Error	Exception
Typically refers to syntax errors (compile-time) – code is malformed.	Occurs at runtime – code is syntactically correct but something goes wrong.
Cannot be handled (program won't run).	Can be caught and handled using <code>try-except</code> .
Example: <code>print "hello"</code> (missing parentheses in Python 3).	Example: <code>10 / 0</code> → <code>ZeroDivisionError</code> .

Note: In Python, syntax errors are a type of `SyntaxError` (which is an exception), but colloquially we distinguish them.

138. Why use `try-except`?

- To gracefully handle runtime errors without crashing the program.
 - To provide meaningful feedback to the user (instead of a raw stack trace).
 - To implement fallback logic (e.g., retry, use a default value).
 - To clean up resources (via `finally`) even when exceptions occur.
 - To separate error-handling code from normal flow, improving readability.
-

139. What happens if an exception isn't handled?

- The exception propagates up the call stack until:
 - It finds a matching `except` block.

- Or it reaches the top level (module) → the interpreter prints the traceback and terminates the program.
 - Unhandled exceptions cause abrupt termination – no further code executes, and resources may not be cleaned up.
-

140. Difference between `finally` and `else`?

`finally`

Executes always – whether an exception occurred or not.

Used for cleanup (closing files, releasing locks).

Runs even after `return/break/continue` (except if `sys.exit()`).

Order: `try` → `else` → `finally` (if no exception) or `try` → `except` → `finally`.

`else`

Executes only if no exception is raised in the `try` block.

Used for code that should run only when the `try` succeeds (avoids putting extra code inside `try`).

Does not run if an exception occurs.

141. Why is `finally` always executed?

Because the Python runtime guarantees that the `finally` block is part of the exception handling protocol – it is executed during stack unwinding before control leaves the `try` statement. This is essential for ensuring resource deallocation (e.g., closing files, DB connections) even if an exception is raised or a `return` occurs.

142. What is `raise`?

A keyword used to explicitly trigger an exception – either a built-in one or a custom one.

Usage:

python

```
raise ValueError("Invalid value")    # raises an exception
```

```
raise                                # re-raise the current exception
```

It's used for input validation, implementing business rules, and re-raising after logging.

143. Why create custom exceptions?

- To make code more expressive – give names to domain-specific errors (e.g., `InsufficientFundsError`).
 - To add extra context (e.g., custom attributes like `balance`, `attempted_amount`).
 - To allow fine-grained handling – catch only your custom exception, not broad categories.
 - To separate business logic errors from system-level exceptions.
-

144. What is exception hierarchy?

All built-in exceptions inherit from `BaseException` → `Exception` → more specific types.

- `BaseException` is the root – includes `SystemExit`, `KeyboardInterrupt`, `GeneratorExit`.
- `Exception` is the base for all regular exceptions – you usually inherit from this for custom exceptions.
- Common subclasses: `ValueError`, `TypeError`, `IndexError`, `KeyError`, `IOError`, `ZeroDivisionError`, etc.
- *Best practice*: Catch `Exception` (not `BaseException`) to avoid intercepting system-level signals.

145. Why catch specific exceptions instead of `except:`?

- Avoids masking unexpected errors – a bare `except:` catches everything, including `KeyboardInterrupt` and `SystemExit`, which can make your program unkillable.
- Better debugging – you know exactly what error you're handling and can log accordingly.
- Preserve the error context – for unexpected exceptions, you want the program to crash (or at least log and re-raise) so you can fix the root cause.
- Allows multiple handlers – you can respond differently to each exception type.

Example:

```
python

try:
    value = int(input())
except ValueError:
    print("Not a number")          # specific
except Exception as e:
    print("Unexpected", e)        # fallback
```

9. File Handling

146. Why close a file after using it?

- Free up system resources – file descriptors are limited; leaving them open can cause "too many open files" errors.
- Flush buffers – ensure all data is written to disk (especially in write modes).
- Avoid locking issues – other processes may not be able to access the file while it's open.
- Prevent data corruption – if the program crashes without closing, buffered data may be lost.

147. Why use `with open()`?

- `with` creates a context manager that automatically closes the file when the block exits – even if an exception occurs.
 - It's cleaner and more readable – no need to explicitly call `.close()`.
 - It guarantees the file is closed (unlike manual `close()` which might be skipped if an exception is raised before it).
 - It's Pythonic and reduces boilerplate.
-

148. Difference between `read()`, `readline()`, and `readlines()`?

Method	Behavior
<code>read(size)</code>	Reads the entire file as a single string (or up to <code>size</code> bytes).
<code>readline(size)</code>	Reads the next line (including trailing newline) as a string, up to <code>size</code> bytes.
<code>readlines(hint)</code>	Reads all lines and returns a list of strings (each line as an element).

Performance: For large files, use `for line in file:` (iterates line-by-line without loading entire file into memory) – it's memory-efficient.

149. Why use binary mode (`'rb'`, `'wb'`)?

- To handle non-text files (images, audio, video, executables, compressed files).
 - Prevents automatic newline translation – in text mode, Python converts `\n` to OS-specific line endings (`\r\n` on Windows), which would corrupt binary data.
 - To read/write bytes objects instead of strings – essential for file formats that aren't text.
-

150. What happens internally when a file is opened?

1. System call `open()` – the OS allocates a file descriptor (integer).
2. Python creates a `FileIO` object (or `_io.TextIOWrapper`) that wraps the descriptor.
3. Buffering is set up (default: line-buffered for interactive, otherwise chunk-buffered).
4. The file pointer is positioned at the start (unless `'a'` mode, then at the end).
5. The Python `file` object is returned, with methods like `read`, `write`, `seek`.

Under the hood: CPython uses the `stdio` library (C) – functions like `fopen`, `fread`, `fwrite`.

Interview Pro-Tips for this final section:

- Q139 (unhandled exception) – mention that `sys.excepthook` can intercept unhandled exceptions for logging.
- Q144 (hierarchy) – note that `Exception` inherits from `BaseException`, and you should never catch `BaseException` unless you have a very good reason.
- Q145 (specific exceptions) – always give the `except:` vs `except Exception` nuance – bare `except:` catches `KeyboardInterrupt`, which is bad.
- Q147 (with `open`) – explain that `open` returns a context manager that calls `__enter__` and `__exit__` – the latter closes the file.
- Q150 (internals) – if they ask, mention *buffering* and *file descriptor* – it shows systems knowledge.

1. Why is a list mutable?

To support dynamic resizing and in-place modifications (`append`, `pop`, `insert`, `del`) without creating a new object each time. This is efficient for collections that change during runtime. Mutability allows you to modify the same object in memory, saving memory allocation overhead.

2. What happens internally when we use `append()`?

1. Python checks if the list has allocated capacity for one more element.
 2. If yes → stores the new element at the next available slot and increments the length.
 3. If no (list is full) → triggers overallocation:
 - Allocates a new, larger array (typically ~12.5% extra capacity in CPython).
 - Copies all existing elements to the new array.
 - Frees the old array.
 - Adds the new element.
 - Updates the list's internal pointers (`ob_item`, `allocated`).
 4. The list's reference count for the new object is incremented.
-

3. Difference between `append()` and `extend()`?

`append(x)`

Adds `x` as a single element at the end.

Length increases by 1.

`extend(iterable)`

Iterates over `iterable` and adds each element individually.

Length increases by `len(iterable)`.

<code>list.append([1,2]) → [[1,2]] (nested).</code>	<code>list.extend([1,2]) → [1,2] (flat).</code>
---	---

Works with any object.

Requires an iterable.

4. Why does `append([1, 2])` create a nested list?

Because `append()` treats its argument as a single object and adds it as-is (without unpacking). The list `[1, 2]` is a single object—it becomes the last element of the outer list. This is how `append()` is designed: it adds exactly one item.

5. Difference between `remove()`, `pop()`, and `del`?

<code>remove(value)</code>	<code>pop(index)</code>	<code>del list[index]</code>
Removes the first occurrence of <code>value</code> .	Removes element at <code>index</code> (default: last) and returns it.	Removes element at <code>index</code> (or slice) – no return.
Searches $O(n)$ by value.	$O(n)$ for arbitrary index (shifts elements) – $O(1)$ for end.	$O(n)$ for arbitrary index (shifts elements).
Raises <code>ValueError</code> if not found.	Raises <code>IndexError</code> if index out of range.	Raises <code>IndexError</code> if index out of range.
Example: <code>lst.remove(5)</code> .	Example: <code>last = lst.pop()</code> .	Example: <code>del lst[0]</code> or <code>del lst[2:5]</code> .

6. Why does `pop()` return a value but `remove()` doesn't?

- `pop()` is used when you need to access and remove an element (common in stack/queue operations). Returning the value is essential.
 - `remove()` is used when you only want to delete a specific value, and you don't need the removed element. Returning it would add unnecessary overhead.
-

7. Difference between `sort()` and `sorted()`?

`sort()` (List method)

`sorted()` (Built-in)

In-place – modifies the original list.

Returns a new sorted list – original unchanged.

Returns `None`.

Returns a new list.

Works only on lists.

Works on any iterable (tuples, dicts, generators).

More memory-efficient (no copy).

Can use more memory (copies).

8. Why does `sort()` return `None`?

For performance and memory efficiency – it performs an in-place sort (mutates the original list) and returns `None` to signal mutation. This prevents chaining (e.g., `lst.sort().append()`), which would be misleading since the original is modified. Returning `None` reinforces that you're modifying the object, not creating a new one.

9. Difference between `copy()` and assignment `=`?

`new = old.copy()`

`new = old` (Assignment)

Creates a shallow copy – a new list object with references to the same elements.

Aliases – both variables point to the exact same list object.

Changes to `new` don't affect `old` (except nested mutable elements).

Changes via one affect the other (they're the same object).

Creates a new list in memory.

No new object – just a new reference.

10. What is a shallow copy?

A new list object is created, but the elements inside are not copied – they are references to the same objects in the original list.

- Changing the top-level list (e.g., `append`, `pop`) doesn't affect the original.
 - But mutating nested mutable objects (e.g., `new[0].append(5)`) will affect the original because both lists share the same inner object references.
-

11. What is a deep copy?

A recursive copy – a new list is created, and every object inside it is also copied recursively (new objects for all nested levels).

- No data is shared – changes in the deep copy never affect the original.
 - Done via `copy.deepcopy()`.
 - More expensive in time and memory than shallow copy.
-

12. Why is list indexing $O(1)$?

Lists in CPython are implemented as contiguous arrays of `PyObject*` pointers.

- Indexing: `lst[i]` calculates the memory address: `base_address + i * sizeof(pointer)`.
 - This is a direct memory access (random access) – no traversal needed, hence $O(1)$.
-

13. Why is searching in a list $O(n)$?

Because Python has no index on values – it must perform a linear scan from start to end until it finds the target or hits the end. In the worst case, it checks all `n` elements $\rightarrow O(n)$ time. Use a `set` or `dict` for $O(1)$ membership tests.

14. Can a list contain another list?

Yes. Lists are heterogeneous and can hold any Python object, including other lists (nested lists). This is useful for matrices, trees, and multi-dimensional data.

15. Can a list contain itself?

Yes, through assignment: `lst = []; lst.append(lst)`.

- This creates a circular reference (list references itself).
 - The list's `repr()` will show `[...]` to indicate recursion.
 - Be careful – circular references require the cyclic garbage collector to clean up.
-

16. What happens if we modify a list while iterating?

Bad things. Modifying a list (adding/removing elements) while iterating over it can cause:

- Skipped elements (if you remove while iterating forward).
- Infinite loops (if you append while iterating).
- IndexErrors (if you delete and the length changes).

Solution: Iterate over a copy: `for item in lst[:]:` or collect items to remove and delete later.

17. Why is slicing creating a new list?

Because lists are mutable and slicing is designed to be safe – it creates a new, independent list object so that modifications to the slice don't affect the original (or vice versa). This aligns with Python's "explicit is better than implicit" philosophy.

18. Difference between `reverse()` and `[::-1]`?

`reverse()`

`[::-1]`

In-place – reverses the original list (mutates).

Returns a new reversed list – original unchanged.

Returns `None`.

Returns a new list.

More memory-efficient ($O(1)$ extra space).

Creates a copy ($O(n)$ memory).

Example: `lst.reverse()`.

Example: `new_lst = lst[::-1]`.

19. Why are lists ordered?

Because elements are stored in a contiguous sequence and accessed by integer indices (0 to n-1). The order reflects the insertion sequence, and it's preserved until explicitly changed. This is fundamental to the list's behavior.

20. Can list elements have different data types?

Yes. Lists are heterogeneous – they hold references to Python objects, and objects can be of any type. There's no type restriction: `[1, "hello", 3.14, [1, 2]]` is valid.

21. Why is list preferred over tuple?

- Mutability – you can add/remove/modify elements.
 - More methods – `append`, `pop`, `insert`, `remove`, `sort`, etc.
 - Dynamic sizing – grows/shrinks as needed.
 - Performance – for frequent modifications, lists are more efficient than creating new tuples repeatedly.
-

22. When should a list not be used?

- When the data is fixed and won't change → use a tuple (immutable, hashable, memory-efficient).
 - When you need fast membership tests ($O(1)$) → use a set.
 - When you need key-based lookups → use a dict.
 - When you need concurrent access (lists are not thread-safe) → consider thread-safe structures.
-

23. Why is `insert` slower than `append`?

- `append` adds at the end – no elements need to be shifted ($O(1)$ amortized).

- `insert(0, x)` inserts at the beginning – all existing elements must be shifted right by one position → $O(n)$. The closer to the front, the more shifting required.
-

24. What happens when a list's memory becomes full?

Python's list uses overallocation – it doesn't grow element-by-element. When the list is full, it:

1. Calculates a new capacity (roughly `new_size = old_size + (old_size >> 3) + 3`).
2. Allocates a new, larger array.
3. Copies all existing element references from the old array to the new one.
4. Frees the old array (decrefs references).
5. Updates the list's internal pointers.

This happens rarely – amortized cost is $O(1)$ per append.

25. Why is `append` nearly $O(1)$?

Because of amortized analysis – the expensive reallocation happens infrequently. The overallocation strategy ($\approx 12.5\%$ extra capacity) ensures that over many appends, the average cost per append is $O(1)$.

26. Can duplicate values exist in a list?

Yes. Lists allow duplicates – they are ordered sequences, not sets. Duplicate values are stored as separate elements at different indices.

27. Why are duplicates allowed?

- Mathematics of sequences – indices are unique, but values don't have to be.
- Practicality – duplicates are common in real-world data (e.g., list of prices, names).
- Flexibility – you can count, filter, or process duplicates as needed.

28. Difference between nested list and 2D list?

Nested List	2D List
A list that contains other lists (possibly of different shapes/sizes).	A specific case of nested list where all inner lists have the same length (rectangular matrix).
May be jagged (irregular).	Is regular (rows and columns).
Example: <code>[[1,2], [3,4,5], [6]]</code> .	Example: <code>[[1,2,3], [4,5,6], [7,8,9]]</code> .

29. How does list iteration work?

Python uses an iterator protocol:

1. `iter(lst)` returns a list iterator object.
 2. The iterator maintains an internal index (starting at 0).
 3. Each iteration calls `next(iterator)`, which returns the element at the current index and increments the index.
 4. When the index reaches `len(lst)`, `StopIteration` is raised.
Under the hood: The iterator accesses the list's internal `PyObject**` array directly.
-

30. What is list comprehension?

A concise syntax for creating lists by applying an expression to each element of an iterable, optionally with a condition.

Example: `[x**2 for x in range(10) if x % 2 == 0]` → `[0, 4, 16, 36, 64]`

31. Why is list comprehension faster?

- Optimized C-level loop – the comprehension is compiled to a specialized bytecode (`LIST_APPEND`) that avoids repeated `append()` method lookup and function call overhead.
 - The loop is executed inside the interpreter in C, not in Python bytecode for each iteration.
 - Usually 1.5-2x faster than a manual `for` loop with `append()`.
-

32. Can a list be a dictionary key? Why?

No. Lists are mutable (unhashable). Dictionary keys must be hashable (immutable and with a constant hash). If a list key were allowed, modifying it would change its hash, making it impossible for the dict to find the value later.

33. Why can't a list be stored in a set?

Same reason – sets require hashable elements. Lists are unhashable because they're mutable, so Python raises `TypeError: unhashable type: 'list'`.

34. What happens internally when `clear()` is called?

1. The list's length is set to `0`.

2. The reference count for each element is decremented (objects may be freed if ref count hits 0).
 3. The internal `ob_item` pointer is set to `NULL` (or a small empty array).
 4. The allocated capacity may or may not be released (it might be retained to allow fast re-use).
 5. Memory is not immediately returned to the OS; it stays with the Python allocator for future list allocations.
-

35. Real-world use case of lists.

- Task queues – store pending tasks (append new tasks, pop tasks for processing).
 - Logging / history – store recent actions or events.
 - Data pipelines – collect results from processing steps.
 - E-commerce carts – store selected items (add/remove/update quantities).
 - Matrix operations – represent matrices as lists of lists (nested).
 - Batch processing – read data from files, process each row, store in a list for aggregation.
 - Undo/redo functionality – maintain a stack of states (using `append()` and `pop()`).
-

Interview Pro-Tips for this section:

- Q2 (append internals) – mention *"overallocation with ~12.5% growth"* – this shows you know CPython internals.
- Q10/11 (shallow vs deep) – always give the nested list example: `a = [[1,2]]`; `b = a.copy(); b[0].append(3)` affects `a` in shallow copy but not in deep copy.
- Q24/25 (capacity and amortized $O(1)$) – explain *overallocation* and *amortized analysis* – it's a classic.
- Q32/33 (list as key/ in set) – mention *hashability* and *immutability* – this is a common trap.
- Q34 (clear) – mention that capacity may be retained (with `list_clear` in CPython), which is an optimization.

1. Why is tuple immutable?

- Hashability – tuples can be used as dictionary keys and stored in sets (hash must remain constant).
 - Integrity – guarantees the data won't change accidentally (e.g., coordinates, configuration).
 - Performance – allows optimizations in memory allocation and iteration (fixed size).
 - Thread-safety – read-only access across threads without locking.
-

2. Why use tuple instead of list?

Reason	Explanation
Data integrity	Fixed data (e.g., days of week, RGB values) that shouldn't change.
Hashability	Need to use as a dict key or set element.
Performance	Slightly faster iteration and less memory.
Semantic clarity	Signals to other developers that the data is immutable and shouldn't be modified.
Return values	Often used for multiple returns from functions (idiomatic).

3. Difference between tuple and list?

Tuple	List
Immutable – cannot change size or elements.	Mutable – can add/remove/modify.
Hashable (if all elements are hashable).	Unhashable.
Smaller memory footprint (no overallocation).	Overallocates capacity.
Slightly faster iteration and access.	Slower iteration (extra overhead for dynamic checks).
Defined with <code>()</code> or comma-separated.	Defined with <code>[]</code> .
No methods like <code>append</code> , <code>remove</code> , <code>sort</code> .	Has many mutating methods.

4. Can a tuple contain mutable objects?

Yes. A tuple can contain lists, dictionaries, or sets. Example: `t = (1, [2, 3], 4)`. The tuple itself is immutable (its references are fixed), but the referenced objects can mutate.

5. If tuple is immutable, how can a list inside a tuple change?

The tuple stores references to objects, not the objects themselves. The references in the tuple are fixed – you cannot point to a different list. However, you can mutate the list object itself (e.g., `t[1].append(5)`), because the reference in the tuple still points to the same list object, and lists are mutable.

6. Why is tuple faster?

- Creation – no overallocation; exact size is allocated once.
 - Iteration – the interpreter knows the size is fixed, so iterating is slightly more efficient.
 - Access – `__getitem__` is simpler (no resizing logic).
 - Memory – less cache misses due to compact layout.
- Benchmarks show tuples are ~10-20% faster for iteration and access.
-

7. Why does tuple consume less memory?

- Tuples allocate exactly the required amount of memory upfront.
 - Lists allocate extra capacity for future growth (~12.5% overallocation).
 - Tuple struct in CPython is smaller (less metadata to maintain).
 - For a tuple of `n` elements, memory is roughly `sizeof(PyTupleObject) + n * sizeof(PyObject*)`, whereas lists have extra allocated slots.
-

8. Can a tuple be a dictionary key?

Yes, provided all elements of the tuple are hashable. If any element is mutable (e.g., a list), the tuple becomes unhashable and cannot be used as a key.

9. Why is tuple hashable?

Because it's immutable – its contents never change, so its hash value (computed recursively from its elements) remains constant throughout its lifetime. This makes it safe for use in hash tables.

10. What makes a tuple unhashable?

If any of its elements is unhashable (e.g., a list, dict, or set). For example: `(1, [2, 3], 4)` is unhashable because the list is mutable. The tuple's `__hash__` method will raise `TypeError` if called.

11. Difference between `(10)` and `(10, )`?

`(10)`

`(10, )`

Parentheses with a single item without comma – evaluates to just the integer `10` (type `int`).

Single-element tuple – type `tuple` with one element `10`.

The parentheses are just grouping, not creating a tuple.

The comma is essential for a one-element tuple.

12. Why is the comma important?

The comma is the tuple constructor, not the parentheses. Parentheses are only for grouping. Without a comma, a single item in parentheses is just that item. The comma tells Python to create a tuple.

13. What is tuple packing?

The process of implicitly creating a tuple when you assign multiple values separated by commas to a single variable:

```
python
```

```
t = 1, 2, 3    # t is (1, 2, 3) - tuple packing
```

14. What is tuple unpacking?

The process of extracting values from a tuple and assigning them to separate variables:

python

```
a, b, c = (1, 2, 3)    # a=1, b=2, c=3
```

Also works with any iterable via `*` (extended unpacking).

15. Can a tuple contain duplicate values?

Yes. Tuples are sequences, not sets – duplicates are allowed. `(1, 1, 2)` is perfectly valid.

16. Why are duplicates allowed?

- Tuples are ordered collections – they store positions, not uniqueness.
 - Duplicates may be meaningful (e.g., coordinates with repeated values, or multiple occurrences in a record).
-

17. Can a tuple be nested?

Yes. A tuple can contain any object, including other tuples: `t = (1, (2, 3), 4)`. This is commonly used for hierarchical data.

18. Difference between `tuple()` and `()`?

`tuple()`

`()`

Constructor – creates an empty tuple.

Literal – also creates an empty tuple.

Both produce the same empty tuple `()`.

The literal is more Pythonic and faster.

`tuple()` can also convert other iterables to tuples (e.g., `tuple([1,2])` → `(1,2)`). `()` is only for an empty tuple.

19. What happens internally when a tuple is created?

1. Python calculates the total size needed: `sizeof(PyTupleObject) + n * sizeof(PyObject*)`.
 2. Allocates memory from the heap (using `PyObject_GC_New`).
 3. Populates the tuple's internal array with references to the provided objects (incref each).
 4. Sets the tuple's length.
 5. The tuple object is returned.
No overallocation – exact size.
-

20. Why is tuple used in database records?

- Database rows are fixed-schema – the number and types of columns don't change.
 - Tuples are lightweight and fast – good for processing many rows.
 - Immutable – prevents accidental modification of row data.
 - Hashable – can be used as keys in dicts for caching or grouping (if all fields are hashable).
-

21. Why do functions return tuples?

- To return multiple values without creating a custom object.
 - It's idiomatic – e.g., `divmod(a, b)` returns `(quotient, remainder)`.
 - Tuples are lightweight and clear – caller can unpack immediately.
-

22. Can a tuple be modified indirectly?

Yes, if it contains mutable objects, you can mutate those objects (e.g., `t = ([1,2],)`; `t[0].append(3)`). The tuple itself still has the same references, but the referenced objects have changed. This is often called "tuple of mutable objects".

23. Why doesn't tuple have `append()`?

Because tuples are immutable – adding an element would require changing the size and allocating a new tuple, which contradicts immutability. Use a list if you need dynamic sizing.

24. Why can't tuple delete individual elements?

Again, immutability – you cannot delete a single element from a tuple. If you need to remove an element, you must create a new tuple (e.g., via slicing).

25. Real-world use case of tuples.

- Geographic coordinates – `(latitude, longitude)`.
- RGB color values – `(255, 128, 0)`.
- HTTP headers – `('Content-Type', 'application/json')`.
- Database rows – result sets from `sqlite3` are tuples.

- Configuration settings – e.g., (host, port, timeout).
 - Multiple return values – return status, message.
 - Dictionary keys – for composite keys: {(city, year): population}.
 - Immutable constant data – days of week, months, etc.
-

 Interview Pro-Tips for this section:

- Q5 (mutable inside tuple) – this is a classic trick question. Always clarify that the *tuple itself* is immutable, but if it contains a list, you can modify the list.
- Q11/12 (comma) – mention that the comma defines the tuple, not parentheses – this surprises many juniors.
- Q19 (internals) – mention *exact allocation, no overallocation* – shows depth.
- Q22 (indirect modification) – frame it as "*the references are fixed, but the referenced objects are not*" – this is a key concept.

1. Why is dictionary called key-value pair?

Because it stores data as mappings from unique keys to associated values. Each entry is a pair: the key is used to compute a hash that determines storage location, and the value is the data you retrieve. This design enables fast lookups by key.

2. Why are dictionary keys unique?

Keys are used as identifiers to retrieve values. If duplicate keys were allowed, there'd be ambiguity about which value corresponds to the key. Python resolves this by keeping only the last inserted value for a duplicate key. Uniqueness is a fundamental property of hash tables.

3. Can dictionary values repeat?

Yes. Values can be anything and need not be unique. Multiple keys can map to the same object, e.g., `{ 'a' : 1, 'b' : 1 }` is perfectly valid.

4. Why can't a list be a key?

Lists are mutable – they can change after being used as a key. If a list key were modified, its hash would change, making it impossible for the dictionary to locate the original value. Python requires keys to be hashable and immutable, so lists are explicitly unhashable.

5. Why can a tuple be a key?

Tuples are immutable – their contents cannot change, so their hash (computed recursively from elements) remains constant. This makes them hashable and safe for use as dictionary keys, provided all their elements are also hashable.

6. What is hashing?

Hashing is a process that takes arbitrary-sized input (like a string, tuple, or number) and maps it to a fixed-size integer via a deterministic hash function. Python uses this integer to decide where to store the key in the dict's internal table. A good hash distributes keys evenly to minimize collisions.

7. How does dictionary search work in $O(1)$?

1. Compute `hash(key)` → integer.
2. Reduce it to the table size `(hash & (size-1))` in CPython) → initial slot index.

3. Check that slot:

- If empty → key doesn't exist.
 - If occupied and the key matches (`key == existing_key`) → found.
 - If occupied but key differs → collision – probe to the next slot (CPython uses open addressing with pseudo-random probing).
 - Repeat until empty slot or match.
Average-case time is $O(1)$ because the table size grows with load and probing stays bounded.
-

8. Difference between `get()` and `[]`?

`dict[key]` (square brackets)

`dict.get(key, default)`

Raises `KeyError` if key is missing.

Returns `None` (or provided `default`) if key is missing – no exception.

Faster for lookups (no function call overhead).

Slightly slower (method call).

Use when you expect the key to exist.

Use when key might be absent and you want a fallback.

9. Why does `get()` not raise `KeyError`?

Because it's designed for safe lookups – you can check for existence without dealing with exceptions, making code cleaner when the absence of a key is a normal, expected case.

10. Difference between `pop()` and `del`?

`pop(key, default)`

Removes the key and returns the value – useful if you need the removed value.

Raises `KeyError` if key missing (unless default provided).

Example: `val = d.pop('a', 0)`.

`del dict[key]`

Removes the key but returns nothing.

Always raises `KeyError` if key missing.

Example: `del d['a']`.

11. Difference between `update()` and assignment?

`update(iterable)`

Merges multiple key-value pairs (from another dict or iterable of pairs).

Example: `d.update({'a':1, 'b':2})`.

`update()` accepts a dict, list of tuples, or keyword arguments.

`d[key] = value` (assignment)

Adds/updates a single key.

Example: `d['a'] = 1`.

Only works for one key.

12. What happens if a duplicate key is inserted?

The last inserted value overwrites the previous one. No error is raised; the dictionary simply updates the mapping. Example: `d = {'a': 1}; d['a'] = 2 → d['a']` becomes 2.

13. Why are dictionaries mutable?

To allow dynamic updates – you can add, remove, or modify key-value pairs after creation. This makes them versatile for data structures that change at runtime (e.g., caches, counters, configurations).

14. What is a hash collision?

When two different keys compute the same hash (or same reduced index after masking). Since a hash table has limited slots, multiple keys may land in the same bucket. Python must handle collisions by probing or chaining; CPython uses open addressing with pseudo-random probing.

15. How does Python handle hash collisions?

- Open addressing – it probes the table by looking at successive slots until it finds an empty one or the matching key.
 - CPython uses a perturb scheme (a variant of linear probing) with a pseudo-random offset to reduce clustering.
 - The probing function: `perturb >= PERTURB_SHIFT; i = (i * 5 + 1 + perturb) & mask`.
 - This ensures good distribution even under high load.
-

16. Difference between `keys()`, `values()`, and `items()`?

Method	Returns
<code>keys()</code>	A view of all keys in the dict (dynamic – reflects changes).
<code>values()</code>	A view of all values.
<code>items()</code>	A view of all key-value pairs as tuples <code>(key, value)</code> .
All are set-like (for keys and items) and support iteration, membership tests, and union/intersection for keys.	

17. What is dictionary comprehension?

A concise syntax to build dictionaries from an iterable:

```
python
```

```
d = {x: x**2 for x in range(5)}
```

```
# {0:0, 1:1, 2:4, 3:9, 4:16}
```

It's faster and more readable than manual loops.

18. Why is a dictionary faster than a list for searching?

- List search: $O(n)$ – checks each element sequentially.

- Dict search: $O(1)$ average – uses hash table to compute the bucket directly, with minimal probing.
For large datasets, the difference is dramatic.
-

19. Difference between shallow copy and deep copy for dicts?

Shallow copy	Deep copy
Creates a new dict but reuses references to the same value objects.	Recursively creates new copies of all nested objects.
Use <code>d.copy()</code> , <code>dict(d)</code> , or <code>copy.copy(d)</code> .	Use <code>copy.deepcopy(d)</code> .
Mutating nested mutable values (e.g., lists) will affect the original.	Nested mutations are independent.

20. Can a dictionary contain another dictionary?

Yes. Values can be any Python object, including dicts. This enables nested data structures (e.g., JSON-like data).

21. Can a dictionary contain a list?

Yes. Lists are valid values. Example: `d = {'numbers': [1, 2, 3]}`.

22. Can a dictionary contain a tuple?

Yes. Tuples can be values (or keys, if hashable).

23. Why are keys immutable?

Because immutable objects have constant hashes. If a key could change, its hash would change, and the dictionary would lose the ability to find the associated value. Immutability guarantees that the key's hash never changes during its lifetime.

24. What happens internally during a lookup (`d[key]`)?

1. Compute hash of `key`.
 2. Reduce hash to table index using mask.
 3. Check the slot at that index:
 - If slot is empty → `KeyError`.
 - If key matches → return value.
 - If key differs → collision → continue probing.
 - Repeat until found or an empty slot is reached.
-

25. Can a dictionary preserve insertion order?

Yes. Since Python 3.7, insertion order is guaranteed (and was an implementation detail in 3.6). Iterating over keys, values, or items returns them in the order they were first inserted.

26. Since which Python version is order guaranteed?

Python 3.7 – it's part of the language specification. It was an implementation detail in CPython 3.6, but officially guaranteed from 3.7 onward.

27. Difference between `OrderedDict` and regular `dict`?

Regular dict (since 3.7)

`OrderedDict`

Remembers insertion order.

Remembers insertion order (same behavior).

Does not have methods like `move_to_end()`.

Has `move_to_end()`, `popitem(last=True/False)` for fine-grained order control.

Equality check ignores order (two dicts with same items but different order are equal).

Equality check considers order (`OrderedDicts` are equal only if order matches).

From Python 3.7, dict order is guaranteed, but `OrderedDict` remains useful for order-sensitive operations (e.g., when serializing to JSON with order).

28. What happens if a key is missing and you use `[]`?

`KeyError` is raised. Use `get()` or `setdefault()` to handle missing keys gracefully.

29. Difference between `setdefault()` and `get()`?

```
get(key, default)
```

Returns the value if key exists, else `default`.

Does not modify the dict.

Safe for read-only lookups.

```
setdefault(key, default)
```

If key exists, returns its value. If key is missing, inserts the key with `default` and returns `default`.

Modifies the dict when key is absent.

Useful for initializing missing keys (e.g., `d.setdefault('count', 0)`).

30. Can a dictionary be nested?

Yes. Dictionaries can contain other dictionaries as values, allowing arbitrarily deep nesting. Common in JSON, configuration files, and hierarchical data.

31. Why does a dictionary not allow indexing (integer index)?

Because dictionaries are mappings, not sequences. They use keys to access values, not positions. Indexing by integer would be ambiguous – it would conflict with the key space (e.g., `d[0]` would mean key `0`, not the 0th element). Use `list(d.keys())[0]` if you need the first key.

32. How is memory allocated for a dictionary?

- CPython uses a compact representation since 3.6:
 - Indices array – sparse table storing integers (indices into entries).
 - Entries array – dense array storing (hash, key, value) triples in insertion order.

- Memory is allocated incrementally; when load factor exceeds 2/3, the table is resized (typically doubling) and re-hashed.
 - This representation saves memory compared to older implementations (more compact).
-

33. Real-world use case of dictionaries.

- Configuration settings – key-value pairs (e.g., `{'host': 'localhost', 'port': 8080}`).
 - Caching – store computed results keyed by input.
 - Counting occurrences – `Counter` or manual dict for frequency analysis.
 - Mapping IDs to objects – e.g., user ID to user profile.
 - JSON data handling – APIs return JSON that naturally maps to dicts.
 - Memoization – store function results for given arguments.
-

34. Why are APIs usually represented as dictionaries?

- JSON is the de facto standard for REST APIs, and JSON objects directly map to Python dictionaries.
 - Dictionaries are flexible – keys are strings, values can be any type (nested structures).
 - They are self-describing – keys indicate what each value means.
 - Easy to parse, modify, serialize, and deserialize.
 - Native integration with Python's `json` module.
-

35. Design a student record system using dictionaries.

A simple design:

```
python
```

```
students = {
```

```

101: {
    'name': 'Alice',
    'age': 20,
    'courses': ['Math', 'Physics'],
    'grades': {'Math': 85, 'Physics': 90}
},
102: {
    'name': 'Bob',
    'age': 22,
    'courses': ['Chemistry', 'Biology'],
    'grades': {'Chemistry': 78, 'Biology': 88}
}
}

# Operations
# Add student
students[103] = {'name': 'Charlie', 'age': 21, 'courses': ['CS'], 'grades':
{'CS': 92}}

# Retrieve student info
info = students.get(101, 'Student not found')

# Update grade
students[101]['grades']['Math'] = 95

# List all students
for sid, data in students.items():

    print(f"{sid}: {data['name']} - Avg:
{sum(data['grades'].values())/len(data['grades']):.2f}")

```

This design uses nested dictionaries for scalability, with fast lookups by student ID.

🎯 Interview Pro-Tips for this section:

- Q7 ($O(1)$) – mention "average-case $O(1)$, worst-case $O(n)$ if many collisions" – demonstrates nuance.
- Q15 (collision handling) – mention "open addressing with perturb" – shows deep CPython knowledge.
- Q24 (lookup internals) – explain the hash $\rightarrow$ mask $\rightarrow$ probe flow.

- Q25–27 (order) – be clear about the 3.7 guarantee and OrderedDict's extra features.
- Q32 (memory) – talk about *compact dict* and *indices/entries arrays* to impress.

SETS

1. Why doesn't a set allow duplicates?

Sets are mathematical collections of unique elements – duplicates violate the set definition. Python enforces uniqueness because the set's hash table uses keys to store elements; inserting the same key again would just overwrite the same slot.

2. How does a set remove duplicates?

1. Compute `hash(element)`.
2. Look up the bucket.
3. If the bucket is occupied and `element == existing_element`, the new element is ignored (no change).
4. If the bucket is empty, the element is stored.

Effectively, duplicates are silently discarded – the set remains unchanged.

3. Why is a set unordered?

Sets are implemented with hash tables – the order of elements in the table depends on their hash values, the table's current size, and the insertion/removal history. This order is arbitrary and can change when the table is resized. Sets do not maintain any insertion order.

4. Why can't a list be inside a set?

Lists are mutable and unhashable. Sets require elements to be hashable so they can be stored in a hash table. Lists are explicitly unhashable (`TypeError: unhashable type: 'list'`).

5. Why can a tuple be inside a set?

Tuples are immutable and hashable (if all their elements are hashable). Their hash remains constant, so they can be safely used as set elements.

6. Difference between a set and a list?

Set	List
Unordered (no index).	Ordered (indexed).
Unique elements only.	Allows duplicates.
Hash-based – $O(1)$ membership.	Sequential – $O(n)$ membership.
Mutable, but no <code>append()</code> (use <code>add</code>).	Mutable, has <code>append</code> , <code>pop</code> , etc.
No indexing (<code>s[0]</code> → <code>TypeError</code>).	Supports indexing and slicing.

7. Difference between a set and a dictionary?

Set	Dictionary
Stores only keys (values are placeholders).	Stores key-value pairs.
<code>{1, 2, 3}</code> .	<code>{'a': 1, 'b': 2}</code> .
No associated values.	Values are explicit.
Membership tests are $O(1)$.	Key lookups are $O(1)$.
Iteration yields elements.	Iteration yields keys (or <code>.items()</code> for pairs).

8. Difference between `remove()` and `discard()`?

<code>remove(element)</code>	<code>discard(element)</code>
Removes the element.	Removes the element.
Raises <code>KeyError</code> if element not found.	Does nothing if element not found (silent).
Use when you expect the element to exist.	Use when you're not sure and want to avoid exceptions.

9. Difference between `pop()` in set and list?

<code>set.pop()</code>	<code>list.pop()</code>
------------------------	-------------------------

Removes and returns an arbitrary element.

Removes and returns the last element (or at given index).

Raises `KeyError` if set is empty.

Raises `IndexError` if list is empty.

No index parameter.

Accepts an optional index.

10. Why does `pop()` return a random element?

Because sets are unordered – there is no "first" or "last" element. The element returned is arbitrary (not truly random, but determined by the internal hash table layout, which is unpredictable). This is useful for destructive iteration, but don't rely on the specific element.

11. What is a union?

The set of all elements present in either set (or both).

Operators/methods: `A | B` or `A.union(B)` → returns a new set.

12. What is an intersection?

The set of elements common to both sets.

Operators/methods: `A & B` or `A.intersection(B)`.

13. What is a difference?

Elements in the first set that are not in the second.

Operators/methods: `A - B` or `A.difference(B)`.

14. What is a symmetric difference?

Elements in either set but not in both.

Operators/methods: `A ^ B` or `A.symmetric_difference(B)`.

15. Why are set operations fast?

Because sets are hash-based – membership and operations like union/intersection check the smaller set and use $O(1)$ lookups. The average complexity is $O(\min(\text{len}(A), \text{len}(B)))$ for operations like intersection (iterating over the smaller set and checking membership in the larger), which is very fast.

16. What is hashing in a set?

When you add an element, Python:

1. Calls `hash(element)` to get an integer.
 2. Uses that hash to find a slot in the internal hash table.
 3. Uses `element == existing` to check for equality (resolves collisions).
- The hash allows $O(1)$ membership tests.
-

17. Can a set contain another set?

No. A set is mutable and unhashable, so it cannot be an element of another set. You'll get `TypeError: unhashable type: 'set'`.

18. Why is a frozenset allowed inside a set?

A `frozenset` is immutable and hashable (provided its elements are hashable). Therefore, it can be stored inside another set – this enables *sets of sets*.

19. Difference between `update()` and `union()`?

`update(iterable)`

`union(iterable)`

In-place – modifies the original set.

Returns a new set – original unchanged.

Returns `None`.

Returns a new set.

Adds all elements from the iterable.

Same addition, but returns a new object.

20. Can duplicates exist in a set?

No. By definition, sets contain only unique elements. Duplicates are automatically discarded.

21. What happens internally when a duplicate is inserted?

1. Compute `hash(duplicate)`.
 2. Find the bucket in the hash table.
 3. If an equal element (`==`) already exists in that bucket, the new object is ignored (its reference count is decremented; no mutation occurs).
 4. No error is raised – it's a no-op.
-

22. Real-world use case of a set.

- Removing duplicates from a list.
 - Membership testing – checking if an element exists (fast $O(1)$).
 - Set operations – e.g., finding common friends in social networks, combining unique tags.
 - Tracking visited items – in web crawling or graph traversal.
 - Unique identifiers – store unique user IDs, IP addresses, or email addresses.
-

23. Why use a set for removing duplicates?

- Simplicity – just `unique = set(my_list)`.
 - Performance – $O(n)$ for the entire list, where each insertion is $O(1)$. Doing it manually with a list would be $O(n^2)$.
 - Less code – no manual loops or conditionals.
-

24. Why is membership test faster in a set?

Because sets use hash tables – membership is $O(1)$ average. Lists use linear scan – $O(n)$. For large datasets, the difference is massive (e.g., 10,000 items: ~ 1 vs 5,000 comparisons).

25. Design a program to remove duplicate names using a set.

```
python
```

```
# Input list with duplicates
names = ["Alice", "Bob", "Alice", "Charlie", "Bob", "David"]

# Remove duplicates (order is NOT preserved)
unique_names = list(set(names))
print(unique_names) # e.g., ['David', 'Alice', 'Bob', 'Charlie']
```

```
# If order matters (preserve insertion order):
unique_ordered = list(dict.fromkeys(names)) # Python 3.7+
print(unique_ordered) # ['Alice', 'Bob', 'Charlie', 'David']

# Or using a loop:
seen = set()
unique_loop = []
for name in names:
    if name not in seen:
        seen.add(name)
        unique_loop.append(name)

print(unique_loop) # Preserves order
```

FROZENSET

1. What is a frozenset?

An immutable version of a set – once created, you cannot add, remove, or change elements. It supports all read-only set operations (union, intersection, difference, etc.) but no mutating methods.

2. Why is a frozenset immutable?

To make it hashable – so it can be used as a dictionary key or as an element of another set. Immutability guarantees its hash remains constant.

3. Difference between a set and a frozenset?

Feature	set	frozenset
Mutability	Mutable	Immutable
Hashable	 No	 Yes (if all elements hashable)
Methods	<code>add</code> , <code>remove</code> , <code>discard</code> , <code>pop</code> , <code>update</code>	Read-only: <code>union</code> , <code>intersection</code> , <code>difference</code> etc.
Can be nested	Cannot be inside another set.	Can be inside a set or as a dict key.
Creation	<code>{1,2,3}</code> or <code>set([1,2,3])</code>	<code>frozenset([1,2,3])</code>

4. Why does a frozenset not have `add()`?

Because it's immutable – adding an element would change its contents and potentially its hash, violating immutability. If you need to add, you must create a new frozenset: `fs = fs.union({new_item})`.

5. Why does a frozenset not have `remove()`?

Same reason – immutability. Removing an element would change the object.

6. Can a frozenset be a dictionary key?

Yes, because it's hashable (provided all its elements are hashable). This is one of its primary use cases.

7. Why is a frozenset hashable?

Its contents are fixed – the hash is computed from its elements at creation and never changes. The hash is cached internally for efficiency.

8. Can a frozenset contain mutable objects?

No. A frozenset can only be created if all its elements are hashable. If you try `frozenset([1, [2,3]])`, it will raise `TypeError: unhashable type: 'list'`. The elements themselves must be immutable/hashable.

9. Can a frozenset be inside another set?

Yes. Because frozensets are hashable, they can be nested inside a regular set, enabling a *set of sets*:

python

```
s = {frozenset({1,2}), frozenset({3,4})}
```

10. Why use a frozenset instead of a set?

- As a dictionary key – e.g., grouping by a set of tags.
- As an element of another set – when you need nested sets.
- To represent immutable constants – e.g., a set of allowed values that shouldn't change.
- For thread-safety – read-only access without locks.

- To guarantee integrity – prevent accidental modification.
-

11. Real-world use case of a frozenset.

- Dictionary keys – grouping data by an unordered collection:
 - python
 - `data = {frozenset(['admin', 'user']): ['read', 'write']}`
 - Storing unique pairs – if you have a set of sets (e.g., social network cliques).
 - Caching – using a set of parameters as a cache key.
 - Immutable configuration – e.g., `ALLOWED_EXTENSIONS = frozenset(['.txt', '.pdf'])`.
-

12. Can a frozenset perform a union?

Yes. `fs1.union(fs2)` returns a new frozenset (or a set if the other operand is not a frozenset). Example: `fs1 | fs2` also works (returns a frozenset if both are frozensets, else a set).

13. Can a frozenset perform an intersection?

Yes. `fs1.intersection(fs2)` returns a new frozenset (or set depending on operands).

14. What operations are not allowed on a frozenset?

All mutating operations are forbidden:

- `add()` → `AttributeError`
- `remove()` → `AttributeError`

- `discard()` → `AttributeError`
 - `pop()` → `AttributeError`
 - `clear()` → `AttributeError`
 - `update()`, `intersection_update()`, `difference_update()`, `symmetric_difference_update()` → all raise `AttributeError`.
-

15. Why was a frozenset introduced?

- To provide a hashable, immutable counterpart to `set`.
 - To enable nested sets (a set of sets).
 - To allow dictionary keys that are sets (e.g., for grouping by an unordered collection).
 - To represent constant sets that shouldn't be modified.
 - It also acts as a clue to developers that the data is intended to be immutable.
-

🔗 Interview Pro-Tips for this section:

- Q2/21 (duplicate insertion) – mention *"the duplicate is simply ignored – no error, no change"*.
- Q3 (unordered) – add *"this is why you never rely on set order in your logic"*.
- Q10 (pop random) – clarify *"arbitrary, not truly random – determined by internal hash layout"*.
- Q19 (update vs union) – give the in-place vs new-set distinction (common interview question).
- Frozenset Q6/7 – highlight *hashability = immutability* – this ties back to dict/set key requirements.

CLASS & OBJECT

1. What is a class?

A blueprint or template for creating objects. It defines attributes (data) and methods (behaviors) that all instances of that class will have. In Python, classes are themselves objects (of type `type`).

2. What is an object?

An instance of a class – a concrete entity that has state (stored in instance variables) and behavior (defined by methods). Each object occupies memory and has its own copy of instance attributes.

3. Why do we create classes?

- Abstraction – hide complexity behind a clear interface.
- Encapsulation – bundle data and operations that work on it.
- Reusability – create many objects from one blueprint.
- Maintainability – changes are localized.
- Model real-world entities – represent customers, orders, etc.

4. Why create objects instead of using only variables?

- Objects group related state and behavior together (cohesion).
- Each object has its own namespace – multiple objects can coexist without interference.
- Objects enable polymorphism and inheritance, which variables alone cannot provide.
- They allow stateful operations – methods can remember and modify the object's internal state.

5. What happens when an object is created (`MyClass()`)?

1. `__new__` (constructor) is called – allocates memory and returns a raw instance.
2. `__init__` (initializer) is called with the instance as `self` – sets up attributes.
3. The initialized instance is returned to the caller.
4. The instance's reference count starts at 1.

6. What is a constructor?

- Strictly, `__new__` is the constructor – it creates the object.
- Colloquially, `__init__` is called the constructor because it initializes the newly created object.

7. Why is the constructor (init) needed?

To initialize the object's state – assign initial values to instance attributes, validate inputs, open resources, or perform any setup required for a valid starting state.

8. What happens if a constructor isn't defined?

Python uses the default `__init__` inherited from `object`, which does nothing. The object is created without any user-defined attributes. You can still add attributes later.

9. Difference between class variable and instance variable?

Class Variable	Instance Variable
Defined inside the class but outside methods.	Defined inside methods (usually <code>__init__</code>) using <code>self</code> .
Shared by all instances of the class.	Unique to each instance.
Accessible as <code>ClassName.var</code> or <code>self.var</code> (if not shadowed).	Accessed via <code>self.var</code> .
Change affects all instances (if changed via class).	Change affects only that instance.

10. Difference between class method and instance method?

Instance Method	Class Method
Takes <code>self</code> as first parameter – refers to the specific instance.	Takes <code>cls</code> as first parameter – refers to the class itself.
Can access/modify instance state.	Can access/modify class state (class variables).
Called on an instance: <code>obj.method()</code> .	Decorated with <code>@classmethod</code> ; called on class: <code>Class.method()</code> .

11. Difference between static method and class method?

Static Method	Class Method
No special first parameter (<code>self</code> or <code>cls</code>).	Takes <code>cls</code> as first parameter.
Cannot access class or instance state (unless passed explicitly).	Can access and modify class state.
Decorated with <code>@staticmethod</code> .	Decorated with <code>@classmethod</code> .
Behaves like a plain function, but belongs to the class's namespace.	Gets the class as the first argument, useful for factory methods.

12. Why is `self` required?

Because instance methods need a reference to the specific instance they are called on. Python passes that reference automatically – `self` is the conventional name. Without it, you couldn't access instance attributes.

13. Is `self` a keyword?

No. It's a convention – you could use `this`, `me`, etc. But `self` is strongly recommended by PEP 8.

14. What does `self` actually refer to?

The current instance of the class – the object on which the method was called. It holds the memory address (reference) of that object.

15. Why can multiple objects share one class?

A class is just a template stored once in memory. Each object created from it gets its own copy of instance attributes but shares the same method definitions and class variables. This saves memory and allows code reuse.

16. What happens internally when an object is deleted?

- When the object's reference count drops to zero (or it's garbage collected), its memory is deallocated.
- If the object defines `__del__`, that method is called before deallocation.
- The object's slots are freed, and the memory is returned to the Python allocator.

17. What is `__init__()`?

A special (dunder) method that Python calls automatically after an instance is created, to initialize the object's state. It's the initializer, not the constructor.

18. Is `__init__()` a constructor?

No – `__new__` is the actual constructor (allocates memory). `__init__` is an initializer that sets up the object after creation.

19. What is `__new__()`?

A static method (special) that is called to create a new instance of a class. It allocates memory, returns a raw instance, and is called before `__init__`. It's rarely overridden except in advanced use cases (e.g., singletons, immutable types).

20. Difference between `__new__()` and `__init__()`?

<code>__new__</code>	<code>__init__</code>
Creates and returns a new instance.	Initializes an existing instance.
Called first.	Called second (after <code>__new__</code>).
Takes <code>cls</code> as first argument.	Takes <code>self</code> (the instance) as first argument.
Must return an instance of the class (or subclass).	Must return <code>None</code> .
Used for controlling instance creation.	Used for setting up instance state.

21. Why is `__new__()` called first?

Because you can't initialize an object that doesn't exist yet. `__new__` creates the raw object; `__init__` then fills it with data.

22. Can an object exist without a constructor?

Yes – if `__init__` is omitted, the object is created by `__new__` (which always exists) and the object exists (empty). `__new__` is always called, even if you don't define it, because every class inherits from `object`.

23. Why use private variables?

To encapsulate internal state – restrict direct access so you can enforce invariants, validate data, or change the internal representation without breaking client code.

24. Difference between `__name__` and `__name__`? (*This is likely a typo – they probably meant `__name` vs `__name__`*)

- `__name` (single underscore? actually double underscore) triggers name mangling (private).
- `__name__` (dunder with trailing underscores) is for magic/special methods (e.g., `__init__`, `__str__`). They are part of Python's protocol and should not be created arbitrarily.

25. What is name mangling?

Python renames attributes that start with `__` (but not `__`) by adding the class name: `_ClassName__var`. This prevents accidental collisions in subclasses and makes the attribute "private" by obfuscation.

26. Can private variables be accessed?

Yes – name mangling is only a name transformation, not true access control. You can access `obj._ClassName__var` directly. Python follows the philosophy "we are all consenting adults".

27. Why use getters and setters?

- Validation – ensure values are within valid ranges.
- Computed attributes – return derived values on the fly.

- Encapsulation – hide internal representation changes.
- Logging/debugging – track access or modifications.
- Lazy initialization – compute attributes only when needed.

28. Difference between object and reference?

Object	Reference
The actual data (chunk of memory) on the heap.	A pointer/address to that object.
Created via <code>__new__</code> .	Stored in variables, passed as arguments.
Example: the list <code>[1,2]</code> (the object).	Example: <code>lst = [1,2]</code> – <code>lst</code> is a reference to the list object.
Multiple references can point to the same object.	Two references may refer to the same object.

29. How many objects can be created from one class?

Unlimited (until memory runs out). Each call to the class (`MyClass()`) creates a new, independent object.

30. Real-world example of a class and object.

python

```
class BankAccount:
    def __init__(self, owner, balance=0):
        self.owner = owner
        self.balance = balance
```

```
def deposit(self, amount):
    self.balance += amount

def withdraw(self, amount):
    if amount > self.balance:
        raise ValueError("Insufficient funds")
    self.balance -= amount

# Objects
alice_acc = BankAccount("Alice", 1000)

bob_acc = BankAccount("Bob")
```

Each account is an independent object with its own state.

OOP - ENCAPSULATION

1. What is encapsulation?

The bundling of data (attributes) and methods that operate on that data into a single unit (class), and restricting direct access to internal state. It hides implementation details and exposes a controlled interface.

2. Why is encapsulation important?

- Security – prevents accidental or malicious modification of internal data.
 - Maintainability – you can change the internal implementation without affecting clients.
 - Modularity – objects are self-contained and decoupled.
 - Validation – control how data is set via setters.
 - Abstraction – users interact with high-level methods, not internals.
-

3. Difference between public, protected, and private?

Access Modifier	Python Convention	Meaning
Public	<code>var</code>	Accessible from anywhere. Default.
Protected	<code>_var</code>	Convention: "internal use only". Not enforced, but indicates it shouldn't be accessed externally.
Private	<code>__var</code>	Name-mangled to <code>_ClassName__var</code> . Strongly discouraged to access externally, but still possible.

4. Can Python truly make variables private?

No – Python does not enforce access restrictions. Name mangling provides obfuscation, not true privacy. It's a convention to discourage external access, but determined code can still reach the mangled name.

5. What is name mangling?

(Repeat of above) – the process of renaming `__var` to `_ClassName__var` to avoid name clashes in subclasses and make it "private" by convention.

6. Why use getter methods?

To retrieve the value of a private/protected attribute while allowing controlled access (e.g., returning a copy, logging access, or performing lazy calculation).

7. Why use setter methods?

To update the value of an attribute with validation, side-effects (e.g., notifications), or transformation (e.g., converting units). They enforce business rules.

8. What is data hiding?

The principle of restricting direct access to an object's internal state, exposing only what is necessary via methods. It's a key aspect of encapsulation that reduces complexity and prevents unintended interactions.

9. Real-world example of encapsulation.

A `BankAccount` class:

- Private attribute `__balance` is hidden.
- Public methods `deposit()` and `withdraw()` provide controlled access.
- Setters validate inputs (e.g., `withdraw` cannot exceed balance).
- The internal representation of `__balance` (e.g., `int` vs `float`) can change without affecting code that uses the class.

10. Design an ATM system using encapsulation.

python

```
class ATM:
    def __init__(self, initial_balance=0):
        self.__balance = initial_balance

    def check_balance(self):
        return self.__balance

    def deposit(self, amount):
        if amount <= 0:
            raise ValueError("Amount must be positive")
        self.__balance += amount
        return f"Deposited {amount}. New balance: {self.__balance}"

    def withdraw(self, amount):
        if amount <= 0:
```

```

        raise ValueError("Amount must be positive")
    if amount > self.__balance:
        raise ValueError("Insufficient balance")
    self.__balance -= amount
    return f"Withdrew {amount}. New balance: {self.__balance}"

# Usage
atm = ATM(500)
print(atm.deposit(200))    # Deposited 200. New balance: 700
print(atm.withdraw(150))  # Withdrew 150. New balance: 550
print(atm.check_balance()) # 550

# atm.__balance → AttributeError (name mangled, but can still access via
# atm._ATM__balance)

```

The `__balance` is private; operations are only possible through the provided methods, which enforce validation.

🎯 Interview Pro-Tips for this section:

- Q20/21 (`__new__` vs `__init__`) – mention that `__new__` is rarely overridden; it's used for immutable types or singleton patterns.
- Q23-27 (private/name mangling) – always emphasize the "consenting adults" philosophy – don't claim Python has true privacy.
- Encapsulation Q4 – be clear: Python has naming conventions, not access modifiers.
- Encapsulation Q10 (ATM) – show how private variables prevent direct manipulation and ensure business rules are enforced.

OOP - INHERITANCE

1. Why do we use inheritance?

- Code reuse – child classes automatically acquire parent methods/attributes.
- Hierarchical classification – models "is-a" relationships (e.g., Dog is an Animal).
- Polymorphism – child objects can be used where parent types are expected.
- Extensibility – override or add behavior without modifying existing code.

2. Difference between parent and child class?

Parent (Superclass/Base)	Child (Subclass/Derived)
The class being inherited from.	The class that inherits from the parent.
Contains general/common attributes and methods.	May add new attributes/methods or override existing ones.
Can be abstract (not instantiated) or concrete.	Inherits all non-private members of the parent.

3. Types of inheritance?

- Single – one child inherits from one parent.
- Multilevel – chain: $A \rightarrow B \rightarrow C$.
- Hierarchical – one parent has multiple children.
- Multiple – a child inherits from more than one parent.
- Hybrid – combination of two or more types (e.g., multiple + multilevel).

4. Why does Python support multiple inheritance?

Python allows a class to inherit from multiple base classes to combine functionalities from different sources (e.g., a FlyingCar can inherit from Car and Airplane). This promotes flexibility and code reuse, but it requires careful handling of method resolution order.

5. What is MRO (Method Resolution Order)?

The order in which Python searches for methods/attributes in a class hierarchy, especially under multiple inheritance. Python uses the C3 linearization algorithm to produce a monotonic, deterministic MRO. You can view it via `ClassName.__mro__`.

6. Why use `super()`?

- To call parent class methods cleanly without hardcoding parent names.
- To ensure each parent is called exactly once in multiple inheritance (avoids diamond problem).
- Makes code maintainable – parent changes don't require updates to child.

7. What happens if `super()` isn't used?

- Parent `__init__` is not called automatically – you must call `Parent.__init__(self)` explicitly.
- In multiple inheritance, this can lead to repeated calls and incorrect initialization order (diamond problem), causing subtle bugs.

8. Difference between single and multilevel inheritance?

Single Inheritance

Multilevel Inheritance

One child inherits from one parent.

Chain of inheritance: $A \rightarrow B \rightarrow C$.

```
class Child(Parent):
```

```
class Child(GrandChild): (where  
GrandChild inherits from Parent)
```

Depth = 2.

Depth > 2.

9. Difference between multiple and hierarchical inheritance?

Multiple Inheritance

Hierarchical Inheritance

One child inherits from two or more parents.

One parent has multiple children.

```
class Child(Parent1, Parent2):
```

```
class Child1(Parent):, class  
Child2(Parent):
```

Combines features from multiple sources.

Creates a tree of specialized subclasses under one base.

10. What is hybrid inheritance?

A combination of two or more types of inheritance – e.g., multiple inheritance + multilevel inheritance (like a diamond). It's common in complex object hierarchies; Python's MRO handles it via C3 linearization.

11. Why avoid excessive inheritance?

- Tight coupling – changes in the parent affect all children (fragile base class problem).
- Deep hierarchies are hard to understand and debug.
- Increased complexity – MRO becomes confusing.
- Violates "favor composition over inheritance" – composition is often a more flexible design.

12. Can child override parent methods?

Yes. A child can define a method with the same name as a parent method; this overrides the parent's implementation. The child can also call the parent's version using `super()`.

13. Can child access private variables?

No, not directly. Private variables (with `__`) are name-mangled to `_Parent__var`, so they are not accessible in the child under the same name. However, the child can still access them using the mangled name (not recommended). Protected variables (`_var`) are accessible.

14. Real-world example of inheritance.

python

```
class Vehicle:
    def __init__(self, make, model):
        self.make = make
        self.model = model

    def start(self):
        return "Vehicle starting"

class Car(Vehicle):
    def __init__(self, make, model, doors):
        super().__init__(make, model)
        self.doors = doors

    def start(self):

        return "Car engine starts with a key"
```

15. Design a vehicle management system using inheritance.

python

```
class Vehicle:
    def __init__(self, reg_no, color):
        self.reg_no = reg_no
        self.color = color

    def move(self):
        return "Vehicle moves"
```

```
class Car(Vehicle):
    def move(self):
        return "Car drives on roads"

class Bike(Vehicle):
    def move(self):
        return "Bike rides on two wheels"

class Truck(Vehicle):
    def __init__(self, reg_no, color, load_capacity):
        super().__init__(reg_no, color)
        self.load_capacity = load_capacity

    def move(self):
        return "Truck carries heavy loads"

# Usage
vehicles = [Car("KA-01", "Red"), Bike("KA-02", "Blue"), Truck("KA-03",
"White", 5000)]
for v in vehicles:

    print(v.move())
```

OOP - POLYMORPHISM

1. What is polymorphism?

The ability of objects of different classes to respond to the same method call in their own specific way. It means "many forms" – you can treat different objects through a common interface.

2. Why is polymorphism useful?

- Code reusability – write generic functions that work with any object that supports a given method.

- Flexibility – add new classes without changing existing code (Open/Closed Principle).
- Simplifies APIs – client code uses a uniform interface regardless of the underlying type.

3. What is method overriding?

When a child class provides a specific implementation of a method that is already defined in its parent. The child's version overrides the parent's. This is a form of runtime polymorphism.

4. What is method overloading?

Having multiple methods with the same name but different parameters (number or type) within the same class. It's compile-time polymorphism in languages like Java/C++.

5. Does Python support method overloading?

Not natively. Python does not allow multiple methods with the same name in the same scope (the last one wins). However, you can simulate it using:

- Default arguments (`def add(a, b=0)`).
- Variable arguments (`*args, **kwargs`).
- `functools.singledispatch` for single-dispatch generic functions.

6. Difference between overriding and overloading?

Overriding	Overloading
Redefines a method in a subclass with the same signature.	Multiple methods in the same class with same name but different signatures.

Runtime polymorphism.	Compile-time (static) polymorphism.
Uses inheritance.	No inheritance required.
Python supports it natively.	Python does not support it natively.

7. What is duck typing?

A programming style where the type of an object is determined by its methods and attributes (what it can do), not by its class. If it walks like a duck and quacks like a duck, it's a duck. In Python, you call methods without checking types – if the object supports the method, it works; otherwise, it fails at runtime.

8. Real-world example of polymorphism.

python

```
class Cat:
    def sound(self):
        return "Meow"

class Dog:
    def sound(self):
        return "Bark"

def make_sound(animal):
    print(animal.sound())
```

```
make_sound(Cat()) # Meow
```

```
make_sound(Dog()) # Bark
```

Both `Cat` and `Dog` respond to `sound()`, allowing the function to work with any object that has that method.

9. Why use polymorphism in large projects?

- Maintainability – new subclasses can be added without modifying existing functions.
- Testability – mock objects can replace real ones for unit testing.
- Scalability – codebases grow without needing to rewrite core logic.
- Design patterns – many patterns (Strategy, Command) rely on polymorphism.

10. Design a payment gateway using polymorphism.

python

```
class Payment:
    def pay(self, amount):
        pass

class CreditCard(Payment):
    def pay(self, amount):
        return f"Paid {amount} via Credit Card"

class PayPal(Payment):
    def pay(self, amount):
        return f"Paid {amount} via PayPal"

class BankTransfer(Payment):
    def pay(self, amount):
        return f"Paid {amount} via Bank Transfer"

def process_payment(payment_method, amount):
    print(payment_method.pay(amount))

# Usage
process_payment(CreditCard(), 100)
process_payment(PayPal(), 200)

process_payment(BankTransfer(), 150)
```

New payment methods can be added without changing `process_payment`.

OOP - ABSTRACTION

1. What is abstraction?

The concept of hiding complex implementation details and exposing only the essential features (interface) to the user. It reduces complexity by providing a clear, high-level API, ignoring irrelevant internals.

2. Why is abstraction needed?

- Simplifies usage – users interact with a clean interface.
- Reduces complexity – internal complexity is hidden.
- Promotes maintainability – implementation can change without affecting clients.
- Enforces a contract – ensures subclasses implement required methods.

3. Difference between abstraction and encapsulation?

Abstraction	Encapsulation
Focuses on what an object does (interface).	Focuses on how it does it (internal state and hiding).
Provides a simplified view.	Bundles data and methods; restricts access.
Achieved via abstract classes/interfaces.	Achieved via private/protected members and getters/setters.
Example: <code>Vehicle</code> has <code>start()</code> – you don't know how.	Example: <code>__engine</code> is private; you manipulate via <code>start()</code> .

4. What is an abstract class?

A class that cannot be instantiated and is intended to be a base for other classes. It may contain abstract methods (declared but not implemented) that subclasses must implement. In Python, use the `abc` module.

5. Why can't abstract classes be instantiated?

They are incomplete – they may have abstract methods with no implementation. Instantiating them would lead to invalid objects missing required behavior. The language prevents that.

6. What is the `ABC` module?

The `abc` (Abstract Base Classes) module in Python provides infrastructure for defining abstract classes and abstract methods. You inherit from `ABC` and use `@abstractmethod` decorator.

7. What is `@abstractmethod`?

A decorator that marks a method as abstract – meaning it has no implementation in the abstract class. Any concrete subclass must override this method. If a subclass does not implement all abstract methods, it also becomes abstract.

8. Can an abstract class have normal methods?

Yes. An abstract class can contain concrete (fully implemented) methods that are inherited by subclasses. This provides shared functionality while still enforcing abstract method implementation.

9. Real-world example of abstraction.

```
python
```

```
from abc import ABC, abstractmethod
```

```
class PaymentProcessor(ABC):
```

```

@abstractmethod
def process_payment(self, amount):
    pass

def log_transaction(self, amount):
    print(f"Logging transaction for {amount}")

class CreditCardProcessor(PaymentProcessor):
    def process_payment(self, amount):
        return f"Processing {amount} via credit card"

class PayPalProcessor(PaymentProcessor):
    def process_payment(self, amount):
        return f"Processing {amount} via PayPal"

```

PaymentProcessor defines the abstraction – you don't need to know how each processor works, just that they implement process_payment.

10. Design an online banking system using abstraction.

python

```

from abc import ABC, abstractmethod

class BankAccount(ABC):
    def __init__(self, account_number, balance=0):
        self.account_number = account_number
        self.balance = balance

    @abstractmethod
    def deposit(self, amount):
        pass

    @abstractmethod
    def withdraw(self, amount):
        pass

    @abstractmethod
    def get_account_type(self):
        pass

```

```

    def check_balance(self):
        return self.balance

class SavingsAccount(BankAccount):
    def deposit(self, amount):
        self.balance += amount
        return f"Deposited {amount} to Savings"

    def withdraw(self, amount):
        if amount > self.balance:
            raise ValueError("Insufficient balance")
        self.balance -= amount
        return f"Withdrew {amount} from Savings"

    def get_account_type(self):
        return "Savings"

class CurrentAccount(BankAccount):
    def deposit(self, amount):
        self.balance += amount
        return f"Deposited {amount} to Current"

    def withdraw(self, amount):
        if amount > self.balance + 1000: # Overdraft limit
            raise ValueError("Overdraft limit exceeded")
        self.balance -= amount
        return f"Withdrew {amount} from Current"

    def get_account_type(self):
        return "Current"

# Usage
accounts = [SavingsAccount("SA001", 5000), CurrentAccount("CA001", 2000)]
for acc in accounts:
    print(acc.deposit(1000))
    print(acc.get_account_type())

    print(acc.check_balance())

```

The abstract class `BankAccount` defines a clear contract (interface) – all account types must implement `deposit`, `withdraw`, and `get_account_type`, while sharing common attributes and the `check_balance` method.

🎯 Interview Pro-Tips for this section:

- Inheritance Q6 – always mention that `super()` follows MRO, not just parent – crucial for multiple inheritance.
- Polymorphism Q7 – duck typing is a Python signature trait; give the "call method without type checks" example.
- Abstraction Q3 – emphasize the difference: encapsulation is about *hiding*, abstraction is about *simplifying*.
- Abstraction Q6–7 – show familiarity with the `abc` module – it shows you know Python-specific tools.